第7章 存储系统

存储系统是构成计算机不可缺少的组成部分，其性能将直接影响计算机的性能。在本章里，在描述存储系统层次结构的基础上，将对计算机存储系统的各部分进行仔细说明。

7.1 存储系统概述

7.1.1 存储系统的层次结构

一台计算机中可能会包括各种存储器，如 CPU 内部的通用寄存器组、cache、CPU 外部的二级和三级 cache、主板上的主存储器、主板外的联机（在线）磁盘存储器以及脱机（离线）的磁带和光盘存储器等。这是一些具体的物理存储器，每一种都有它们具体的技术指标。

将上述两种或两种以上的存储器用硬件、软件（或硬件与软件组合）连接在一起并对它们进行管理就构成了存储器系统。例如，cache 存储器与主存储器连接在一起就构成了 cache 存储系统；主存储器与大容量的磁盘存储器构成了虚拟存储系统。现在的 PC 中的存储系统是由 cache 存储系统和虚拟存储系统构成的。

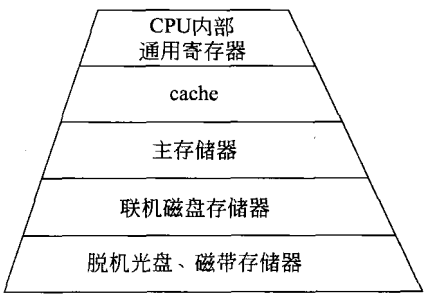


图 7-1 存储系统的层次结构

在计算机中，各种存储器的容量、速度、访问方式和用途等都不相同，这些存储器形成一种层次结构。在这样的层次结构中，不同层次上的存储器发挥不同的功能和作用。共同使计算机高效地工作。计算机存储系统的层次结构如图 7-1 所示。

在图 7-1 所示的计算机层次结构中，越是靠近上层，其速度越快、容量越小而单位存储容量的价格也越高。通过这种层次结构可使计算机的存取速度接近 CPU 的速度，使程序员所使用的主存容量接近联机外存的容量，使整个存储系统的单位存储器价格接近联机外存的价格。

7.1.2 存储器分类

由于构成存储器的材料、性能及使用方法不同，从不同的角度考虑，就有不同的分类方法。

1. 按照存储信息介质分类

目前，在计算机中用于存储信息的介质有半导体器件、磁性器件或光学器件及材料。因

此，就有半导体存储器，磁盘(磁带)存储器，光盘存储器等。

2. 按在计算机中的用途分类

在计算机中用于存放当前正要执行或刚执行的程序和数据的存储器是主存储器，也称为内部存储器，简称内存。

用于存放正在执行程序或正在使用数据，用以克服主存储器速度太慢的存储器是高速缓冲存储器 cache。

在 CPU 内部，用于存放微程序的存储器是控制存储器。

用来存放当前暂不使用的大量信息的存储器是外部存储器，又称外存。如磁盘存储器、光盘存储器、U 盘、MMC(SD)存储卡、磁带存储器等。

3. 按存放信息的易失(挥发)性分类

当存储器断电后，有的存储器所存信息也随之丢失。这类存储器所存信息是易失的、是可挥发的。例如，半导体随机读写存储器 RAM 就是易于挥发的易失存储器。

还有一些存储器在断电后所存信息不会丢失，当再次加电后它所存储的信息依然存在。例如，后面将要讲述的磁盘存储器、各种类型的半导体 ROM 等都是非易失存储器。

4. 按存取方式分类

当存储器的存取时间与存储单元的物理地址无关，随机读写其任一单元所用的时间都一样，这就是随机读写存储器 RAM。

有的存储器的存取只能按顺序进行，这就称为顺序存储器，如磁带存储器。

5. 按存储器的读写功能分类

有的存储器既能读出又能写入，称为读写存储器。而有的存储器的内容已固定，工作时只能读出，称为只读存储器 ROM。

7.1.3 存储器的性能指标

前面已经提到，存储器和存储系统是不同的两种概念，这里说明存储器的主要性能指标。

存储器的性能指标是用来描述存储器性能的，主要包括如下几个方面。

1. 存储容量

这里指的是存储器所能存储的二进制信息的总位数，其表示方式一般为：存储器总存储单元数×每个存储单元的位数。例如，某计算机内存容量为 1024K×8b 或 1024KB、1GB(通常用 b 表示位；B 表示字节)，某磁盘存储器的容量为 160GB。

同样，也用上述方式来描述一块存储器芯片的容量。例如，6264 静态 RAM 的容量为 8K×8b；动态 RAM 芯片 NMC41257 的容量为 256KB。

2. 存储器的速度

(1) 存取时间

存取时间又称访问时间，就是对存储器中某一个单元的数据进行一次存(取)所需要的时间。

对于主存而言，目前都是由半导体存储器构成。人们可以从厂家的技术手册上得到它的存取时间。值得注意的是 CUP 在读写存储器时，它所提供给存储器的读写时间必须比存储器所要求的存取时间长。如果不能满足这一点，计算机将无法正常工作。

厂家已生产出各种速度的存储器芯片，从 1000ns 到 0.5ns，各种存取速度的芯片可任意选择。

(2) 存取周期

存取周期是指连续对存储器进行存(取)时，完成一次存(取)所需要的时间。

存取周期与存取时间很可能不一样。经常是存取周期要大于存取时间。因为在连续存取时，每读出(或写入)一个存储单元，在对下一个存储单元操作前，需要一定的稳定时间。

(3) 存储器带宽

单位时间里存储器可以读出(或写入)的字节数。若存储器的存取周期为 t_m，且每次可读出(或写入)n 个字节，则存储器的带宽 $B_m = n/t_m$。

例如，某计算机主存的存取周期为 t_m 为 10ns，每次可读写 8 个字节。则该计算机主存带宽为 800MBps。

3. 可靠性

计算机要正确地运行，必然要求存储器系统具有很高的可靠性。内存的任何错误就足以使计算机无法正常工作。

对计算机的存储器来说，有的是不可维修的，如构成存储器的超大规模集成电路芯片。一旦出现故障就丢弃换新的，它们是不可维修的。而有的存储器是可维修的，如磁盘存储器、磁带存储器等。可维修部件的可靠性可用平均故障间隔时间(MTBF)来描述，而不可维修部件的可靠性常用平均无故障时间或平均故障前时间(MTTF)来描述。

有时就用 MTBF 来说明存储器的可靠性，例如某 80GB 的硬磁盘存储器的 MTBF 为 30 000 小时。某 SRAM 芯片的 MTBF 为 5×10^6 小时。

除了用 MTBF(或 MTTF)来描述存储器的可靠性外，还有许多其他指标可以说明存储器的可靠性，这里不再介绍。

4. 功耗

使用功耗低的存储器构成存储系统，不仅可以减少对电源容量的要求，而且还可以提高存储系统的可靠性。

5. 价格

构成存储系统时，在满足上述要求的情况下，应尽量降低存储器的价格。通常是以每 KB 或每 MB 的价格来衡量存储器的成本。随着技术的发展，存储器的价格已大大降低。

例如，20 世纪 80 年代初每 MB 需几千元而今天不值一毛钱。除上述指标外，其他如体积、重量、封装方式、工作电压、环境条件等不再说明。

总之，大容量、高速度、高可靠性、低功耗、低成本是存储器的主要要求。

7.2 内部存储器(主存储器)

在本节中将描述与计算机内存有关的问题，包括主存的构成、高速缓存及虚拟存储器等。

7.2.1 随机读写存储器 RAM

现在构成计算机主存的毫无例外的都是半导体存储器，下面将逐一加以介绍。

1. 静态随机读写存储器 SRAM

在计算机中，常用的随机读写存储器 RAM 分为两大类：一类是静态随机读写存储器 SRAM；另一类是动态随机读写存储器 DRAM。下面将分别加以说明。

(1) 内部译码结构

RAM 的生产厂家已为使用者开发研制出各种容量、速度、材料工艺、封装形式的 IC 存储器芯片，其型号品种有数十万种之多，为选用提供了很大的方便。

一种 RAM 芯片，其外部有地址引线、数据引线和控制信号引线。地址引线在芯片内部译码，选中芯片内部的相应存储单元。例如，某静态 RAM 芯片上有 n 条地址线时，这些地址线上所能表示的地址编码就有 2^n 种。这就意味着该芯片内部有 2^n 个存储单元。

为了能对芯片内部的存储单元进行读写，就需要在芯片内部对地址信号进行译码，以便选中地址所对应的存储单元。芯片内部的地址译码有两种方式。

① 一维译码。对于容量很小的芯片，如容量在几百个存储单元以内的芯片，多用一维译码。芯片内部只用一个地址译码器，其译码输出可选中相对应的存储单元。其示意图如图 7-2(a)所示。

② 二维译码。当芯片容量在几百个存储单元以上时，再用一维译码将会使译码器过于复杂。因此，多用二维译码。此时，芯片内部用行译码器和列译码器两个地址译码器分别对地址译码，行译码输出可选中相对应的一行存储单元；而列译码输出可选中相对应的一列存储单元，而行列交叉点上的存储单元就是最终所选中的存储单元。其示意图如图 7-2(b)所示。

由图 7-2(a)可以看到，当芯片容量很大时，地址线会很多，译码器会很复杂。例如，某芯片容量为 64KB，芯片上的地址线就有 16 条——A0～A15。利用一维译码，则译码器必须有 65 536 条译码输出，分别选择 64KB 个存储单元。若采用图 7-2(b)所示的二维译码方式，可将 16 位地址分成 8 条行地址和 8 条列地址，即图中 n 和 m 分别为 8。则行、列地址译码器均为 8 位译码器。行、列地址译码器分别译出 256 行和 256 列。当行、列译码分别有效便可选中一个存储单元。显然，制作两个 8 到 256 译码器比制作一个 16 到 65 536 的译码器要容易得多。

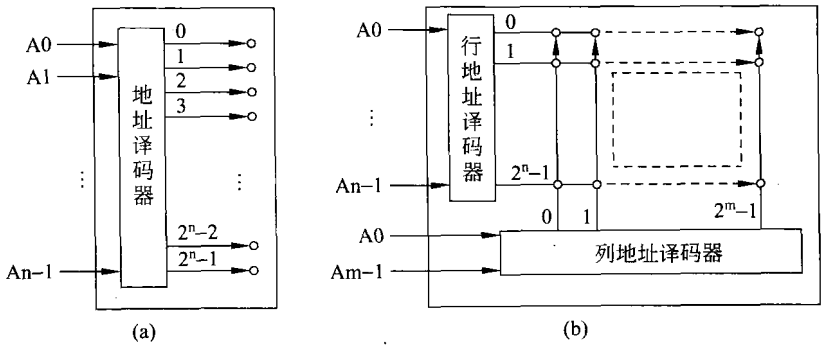


图 7-2　RAM 芯片内部的两种译码方式

(2) 单元电路

在这里将要说明 RAM 中 1b 信息是如何存储的，从而加深对存储器工作原理的理解。

① 静态读写存储器 SRAM

首先，描述 1b 静态 RAM 的构成及工作原理。静态 RAM 可有多种结构，图 7-3 是一种典型的 1 位 SRAM 的构造形式。

图 7-3 中，T3、T4 为 T1、T2 的负载，分别相当于一个电阻。由它们构成了一种双稳态电路，即其 a 点或 b 点均有两种稳定状态，0 或 1。并且 a 和 b 的状态刚好相反，故可以存放 0 或 1。

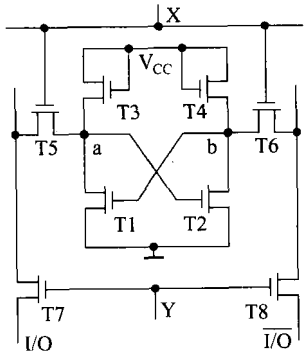


图 7-3　静态 RAM 单元电路

T5、T6、T7、T8 是 4 个用于控制的晶体管，它们的栅极上分加上行译码信号 X 和列译码信号 Y。当行、列译码信号 X 和 Y 同时有效时，便选中了该单元。

写数据时，X、Y 有效，T5、T6、T7、T8 这 4 个晶体管导通。写 1 时，I/O 端上加高电平而$\overline{I/O}$端上加低电平，双稳电路使 a 点为高电平，b 点为低电平。而且，这种状态在写完之后会一直保持下去；写 0 也是一样，只是使 a 点为低电平，b 点为高电平。同样，这种状态在写完之后会一直保留下去。

在读出数据时，经行列地址译码使 X、Y 有效，T5、T6、T7、T8 使 4 个晶体管导通。此时，本单元电路存储的信息——a 点或 a、b 的状态便可以从 I/O 或$\overline{I/O}$上读出。

从上面 1 位 SRAM 单元电路中可以看出以下几点。

a) 图 7-3 只画出 1b 的存储电路，若 n 个这样的电路可构成 n 位存储单元。

b) 数据一旦写入，其信息就稳定的保存在电路中，静静地等待读出。无论读出多少次，只要不断电，此信息会一直保持下去。这也许就是静态的由来。

c) SRAM 初始加电时，其状态是随机的。写入新的状态，原来的旧状态就消失了。新状态一直维持到写入更新的状态为止。

d) 在电路工作时，即使不进行读写操作，只要保持在加电状态下，电路中就一定有晶体管导通，就一定有电流流过，也就一定会有功率消耗。因此，与后面的 DRAM 比较，SRAM 的功耗大，集成度不能做得很高。

② 动态读写存储器 DRAM

动态存储器 DRAM 也有多种结构形式，其中一种如图 7-4 所示。利用所给出的这种结构可阐明 DRAM 的工作原理。

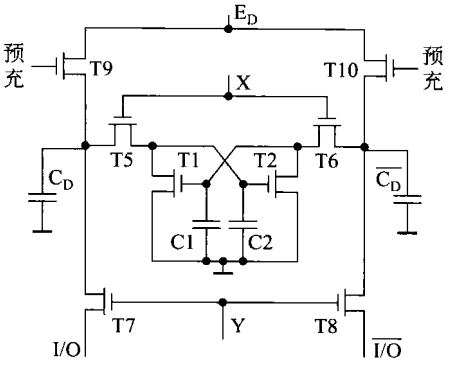


图 7-4 动态 RAM 单元电路

写入数据时，要写入的状态加到 I/O 端上（反相信号加到$\overline{I/O}$端上）。同时，经行列地址译码使 X、Y 有效，T5、T6、T7、T8 使 4 个晶体管导通。写 1 时，对 C2 充电，C1 不充电；写 0 时，对 C1 充电，C2 不充电。写后，状态就保持在电容上。但是，由于 C1、C2 的电容量很小，信息仅能在电容上保持不长（几 ms）的时间。

读出数据时，先产生预充电信号使 T9、T10 导通，由 E_D 对电容 C_D 和$\overline{C_D}$充电。而后，经行列地址译码使 X、Y 有效，T5、T6、T7、T8 使 4 个晶体管导通。此时，若原状态为 1，则 C2 有电荷 T2 导通，预充电的$\overline{C_D}$经 T6、T2 放电，$\overline{I/O}$输出为 0；同时，由于 C1 未充电，T1 截止，则 C_D 上的预充电作为高电平由 I/O 输出。同时还通过 T5 给 C2 补充电荷。

刷新，这是 DRAM 所特有的问题。由上述可见，DRAM 的信息是存放在电容上的，且电容量很小（容量越小速度越快）。尽管不工作时 T1、T2 均不导通，但电容总有很微小的漏电，时间长了，存放的信息就会丢失。因此，必须在电荷漏掉前不断地补充电荷保证信息不会丢失，这就是刷新。刷新一般每 2～4ms 进行一次。

刷新过程就是每次刷新一行，行地址译码输出使 X 有效。由图 7-4 可以看到，X 有效，使 T5、T6 导通。同时，加上预充电信号使 T9、T10 导通。这时电源 E_D 就会对原先充有电荷的 C1 或 C2 充电，补充电荷，而原先未充电的电容不会被充电。从而实现刷新。

根据上述描述，对 DRAM 应注意的是以下几点。

a) 图 7-4 只画出 1b 的存储电路，若 n 个这样的电路可构成 n 位存储单元。

b) DRAM 必须定时不断地刷新，以保证所存储的信息不会丢失。这大概就是将这种类型的 RAM 叫做动态存储器的原因。

c) DRAM 初始加电时，其状态是随机的。写入新的状态，原来的旧状态就消失了。新状态一直维持到写入更新的状态为止。

d) 在电路加上电源不进行读写及刷新操作时，只是保持在加电状态下，电路中没有晶体管导通，也就没有电流流过（会有极微小的漏电流存在），也就没有功率消耗（或功耗很小）。因此，DRAM 的功耗非常小，其集成度可做得很高，当前的一块 DRAM 芯片的集成度达 4GB。而厂家将它们堆叠在一块封装中可达 72GB。

利用上述单元电路，采用微细加工技术，可以生产出各种容量的集成电路芯片，用以构成存储器系统。

2. 主存储器的组成及接口

(1) 存储器与 CPU 速度上的协调

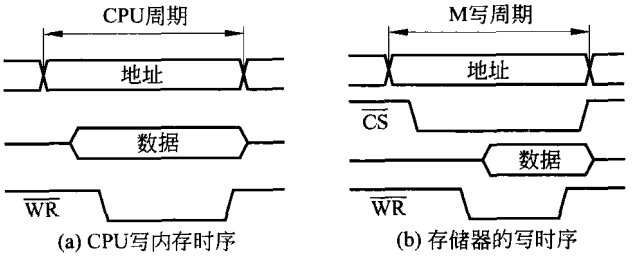
对于简单的计算机，只要将半导体集成电路芯片接在计算机的系统总线上便可以构成

内部存储器。在构成内存时必须注意存储器与 CPU 在速度上的协调。只有当 CPU 读写内存的时间大于内存所要求的读写时间时，才能保证可靠的读写。

在前面的章节中已经看到，CPU 读(或写)内存时，由 CPU 内部的时序电路和控制器产生的控制信号，形成 CPU 读(或写)内存 CPU(或总线或机器)周期，其写内存时序如图 7-5(a)所示。

在 CPU 写内存的周期里，由 CPU 送出内存地址、送出要写入内存地址的数据、还送出写控制信号$\overline{WR}$。这些信号加到系统总线上，传送到要写的内存芯片上。用 CPU 提供的这些信号去实现对存储器的写。不同的 CPU，写内存的 CPU 周期不一样；同样的 CPU 如果时钟频率不一样，CPU 周期也不一样。例如，5MHz 时钟下的 8086 CPU 的写内存 CPU 周期为 4 个时钟周期，即 800ns。

另一方面，厂家研制了数以十万计的各种存储器芯片，前面已经说明了存储器的技术指标。图 7-5(b)画出了存储器的写时序。请读者注意，这是存储器所要求的。不同类型的存储器芯片，写周期的长度是很不一样的，这是存储器芯片所固有的。笔者曾经用过 450ns、180ns、120ns、75ns 和 25ns 的 SRAM 芯片。



(a) CPU写内存时序　　(b) 存储器的写时序

图 7-5　CPU 写内存及内存写的时序

这里要强调的是，在利用存储器芯片构成内存时，必须保证 CPU 的写(或读)内存的周期应大于等于存储器芯片所要求的写(或读)时间。也就是说 CPU 提供的信号持续时间必须大于等于存储器芯片所要求的信号时间。

在实际工程上构成内存时，要考虑到 CPU 发出的信号要经过总线传到存储器芯片上，信号在总线上传送总需要时间(比光速慢得多)。而且信号有可能经过驱动门或缓冲门，而任何门总对信号有延时。考虑上述因素，在选芯片时应留有 30%的容量。例如，若 CPU 对内存的读写周期为 100ns，则所选内存芯片的存取周期不要大于 70ns。

如果工程上不能满足上述要求，也就是快速的 CPU 使用了慢速的存储器时，就需要采取措施：降低 CPU 时钟频率，即延长 CPU 所提供的读写时间；或选用读写速度更快的存储器芯片；或在 CPU 读写该慢速存储器时，适当延长 CPU 的读写时间，常用 READY 信号使 CPU 插入若干个时钟周期。

(2) 内存构成

将存储器芯片连接在系统总线上，便可构成所需内存。不同 CPU 构成的系统总线信号会有所不同。为更方便于系统设计，前人已制定上百种系统总线标准。为了叙述方便，在下面的描述中多使用 PC/XT 或 ISA 总线信号(详细情况见第 8 章)。

① 单块存储器芯片的连接

现以一块典型的 8K×8b 的 CMOS SRAM 6264(或 6164)芯片为例说明其外部特性及

工作过程。该芯片的引脚图如图 7-6 所示。

a) 引线功能

6264(6164)有 28 条引出线，它们包括如下几种。

$A_0 \sim A_{12}$为 13 条地址信号线。芯片上的这 13 条地址线决定了该芯片有多少个存储单元。因为 13 条地址线上的地址信号编码最大可以到 8192(8K)个。也就是说，芯片上的 13 条地址线上的信号经过芯片的内部译码，可以决定 8K 个单元。这 13 条地址线在使用时通常接总线的低位地址，以便 CPU 来寻址该芯片拥有的 8K 个单元。

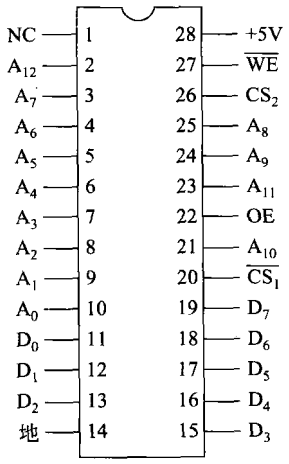


图 7-6 6264 引脚图

$D_0 \sim D_7$为 8 条双向数据线。正如上面所说，静态 RAM 芯片上的地址线的数目决定该芯片有多少个存储单元；而芯片上的数据线的数目决定了芯片中每个存储单元存储了多少个二进制位。6264 有 8 条数据线，说明该芯片的每个单元存放一个字节。在使用中，芯片的数据线与总线的数据线相连接。当 CPU 写芯片的某个单元时，将数据传送到该芯片内部这个指定的单元。当 CPU 读某一单元时，又能将被选中芯片该单元中的数据传送到总线上。

$\overline{CS_1}$，CS_2为两条片选信号的引线，当两个片选信号同时有效时，即$\overline{CS_1}=0$，$CS_2=1$时，才能选中该芯片。不同类型的芯片，其片选信号多少不一，但要选中芯片，只有使芯片上所有片选信号同时有效才行。在使用中，通过对高位地址信号和控制信号的译码产生(或形成)片选信号，把芯片的存储容量放在设计者所希望的内存空间上。简言之，就是利用片选信号将芯片放在所需要的地址范围上。

$\overline{OE}$为输出允许信号。只有当$\overline{OE}=0$，即其有效时，才允许该芯片将某单元的数据送到芯片外部的 $D_0 \sim D_7$ 上。

$\overline{WE}$是写允许信号。当$\overline{WE}=0$时，允许将数据写入芯片；当$\overline{WE}=1$时，允许芯片的数据读出。以上 4 个信号的功能如表 7-1 所示。NC 为没有使用的空脚。芯片上还有+5V 电压和接地线。

表 7-1 6264 真值表

$\overline{WE}$	$\overline{CS_1}$	CS_2	$\overline{OE}$	$D_0 \sim D_7$
0	0	1	×	写入
1	0	1	0	读出
×	0	0	×	
×	1	0	×	三态(高阻)
×	1	1	×	

注：×表示不考虑。

b) 6264(6164)的工作过程

从表 7-1 可以看到，写入数据的过程是：在芯片的 $A_0 \sim A_{12}$上加上要写入单元的地址；在 $D_0 \sim D_7$ 上加上要写入的数据；使$\overline{CS_1}$和 CS_2 同时有效；在$\overline{WE}$上加上有效的低电平，此时$\overline{OE}$可为高也可为低。这样就将数据写到了地址所选中的单元中。

从芯片中某单元读出数据的过程就是：$A_0 \sim A_{12}$加上要读出单元的地址；使$\overline{CS_1}$和CS_2同时有效；使$\overline{OE}$有效(为低电平)；使$\overline{WE}$为高电平，这样即可读出数据。

以上读出或写入过程，实际上是CPU发出信号加到存储器芯片上的一个过程。其时间应满足芯片的要求。

c) 连接

对于使用人员来说，在了解存储器芯片的外部特性之后，重要的是必须掌握存储器芯片与总线的连接，即按照用户的要求，主要是按规定的内存地址范围，将存储器芯片正确地接到总线上，构成所要求的内存。

图7-7就是利用6264构成地址从F0000H到F1FFFH这8KB的内存空间。图7-7连接的是8088最大模式下的8位系统总线PC/XT。

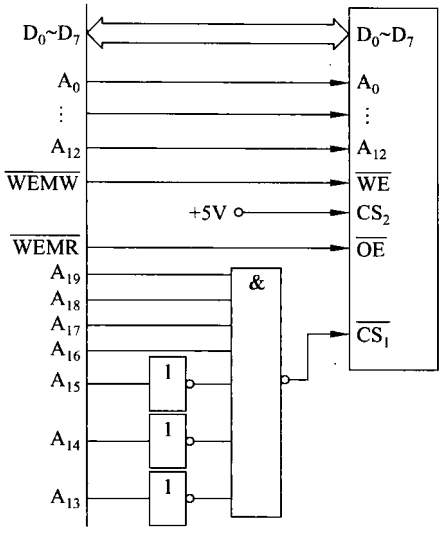


图7-7 6264全地址译码器

② 内存的字扩展

当用一片存储器芯片构成内存的字容量不够时，可采用字扩展的方法达到目的。

如图7-8所示是利用6264芯片，通过字扩展的方法，构成32KB的连接图。

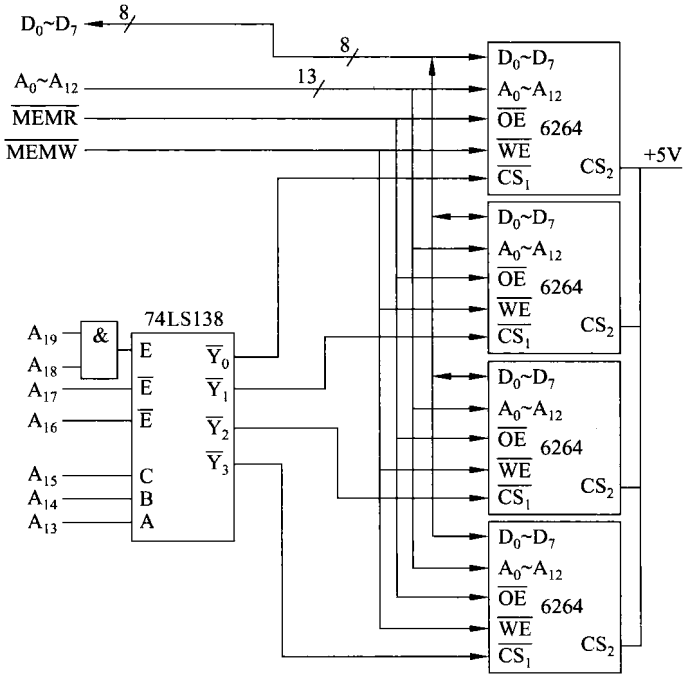


图7-8 内存字扩展的连接电路图

由于6264芯片的容量只有8K×8b，而要构成的内存为32K×8b，这就需要将8KB扩展为32KB。因此，需要4片6264芯片。构成的内存地址为C0000H到C7FFFH。

③ 内存的位扩展

当存储器芯片的数据位数较少，而构成的内存每一存储地址单元要求的存储数据位较多时，可采用位扩展，即用多片存储器芯片来达到位扩展的目的。图 7-9 就是一种内存位扩展连接的形式。

在图 7-9 中，存储器芯片的容量是 2K×4b 的芯片，而要构成的内存是 2KB。因此，需要将芯片的 4 位扩展为内存所需要的 8 位。

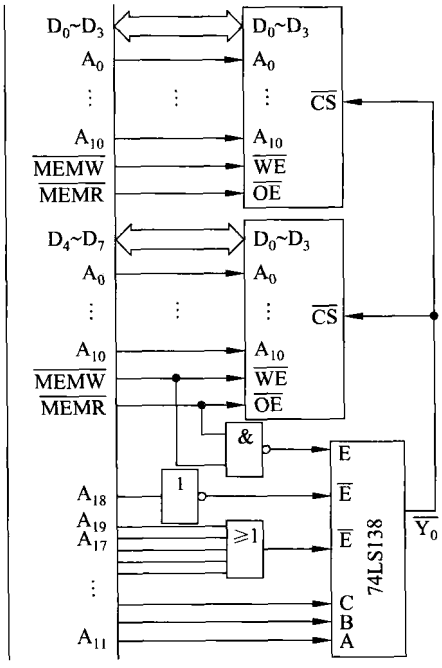


图 7-9 内存位扩展连接

利用二片存储器芯片，其中一片的数据线 $D_0 \sim D_3$ 接系统总线的数据总线 $D_0 \sim D_3$；而另一片的数据线 $D_0 \sim D_3$ 接系统总线的数据总线 $D_4 \sim D_7$。在图 7-9 中，利用 3-8 译码器 74LS138 的译码输出端 $\overline{Y_0}$ 作为两块芯片的片选信号。译码器 $\overline{Y_0}$ 有效时，可同时选中这两块芯片，实现对一个字节 8 位数据的读写。

根据图 7-9 的译码器连接，图中两块芯片所构成的内存地址为 40000H～407FFH，即两片 2K×4b 的芯片构成了 2KB 的内存空间。

④ 80x86 内存的连接

以上在内存连接中都是认为内存一个地址单元存放一个字节，也就是说内存是按字节编址，每一个内存地址就存放 8 位二进制数。

从 16 位的 8086 CPU 开始，80x86 由 16 位到 32 位一直发展到 64 位。在这样的 CPU 构成的系统中，由于 CPU 的特性决定存储器的结构比较复杂。在存储器的连接上也有许多不同，下面将一一予以说明。

a) 8086 的内存接口

8086 CPU 是真正的 16 位处理器，它既能按字节访问内存又能按字(16 位)访问内存。

8086 的系统总线的数据线是 16 位的，即 $D_0 \sim D_{15}$。

(a) 8086 系统中内存奇偶分体

在 8086 系统中，为了能够实现一次访问一个字(16 位)，又能一次访问一个字节(8 位)，故将其内存地址空间分成偶存储体和奇存储体。前者对应地址为偶数而后者对应的地址为奇数。整个 1MB 的内存空间就分为偶存储体 512KB 和奇存储体 512KB。在 8086 系统中，无论实际构成的内存有多大，为了保证 CPU 能一次读写数据的高字节、一次读写数据的低字节或一次读写数据的高低两个字节(即一个 16 位的字)，内存必须分成奇、偶两个体。例如，若构成的内存为 64KB，则必须分成偶地址体 32KB 和奇地址体 32KB。

同时，为做到上面所描述的奇偶分体，在 8086 的引线上，增加了一个$\overline{BHE}$信号。当 8086 读写偶地址字节时，$A_0=0$，$\overline{BHE}=1$；当读写奇数地址字节时，$A_0=1$，$\overline{BHE}=0$，当读写一个 16 位字时，$A_0=0$，$\overline{BHE}=0$。

(b) 连接

在 8086 内存的具体连接上就体现出上述的特点。作为例子，将两片 6264 接到 8086 的系统总线上，构成 16KB 的内存，其连接如图 7-10 所示。

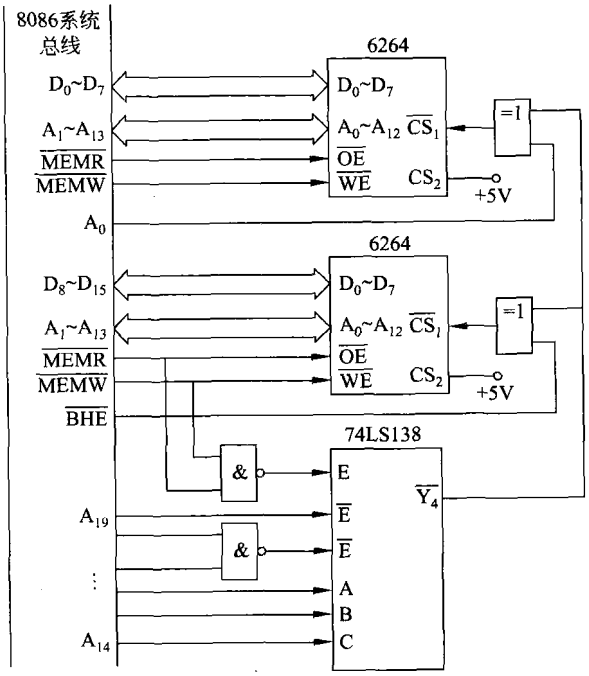


图 7-10　6264 与 8086 系统总线的连接

图 7-10 中，将 16KB 的内存分为两个体，偶地址 8KB，利用高位地址 $A_{19} \sim A_{14}$ 和 A_0 译码构成片选信号；而奇地址 8KB 利用 $A_{19} \sim A_{14}$ 和$\overline{BHE}$译码构成片选信号。形成的内存地址范围为 70000H～73FFFH。

b) 80386、80486 的内存接口

80386 和 80486 都是典型的 32 位处理器。在这里不准备仔细介绍这些处理器，只说明在这样的处理器构成的系统中，内存接口的特点及其实现。

(a) 与内存接口相关的信号

在80386、80486构成的系统中，与内存有关的信号主要有如下几种。

- 地址信号 A_2 到 A_{31} 共30个地址信号，其编码可寻址1G个32位的存储单元。这里没有 A_0 和 A_1，这两个信号已在80386、80486内部译码产生4个体选择信号。
- 体选择信号$\overline{BE0}$到$\overline{BE3}$。与前面提到的8086相类似，在32位的80386、80486系统中，为了灵活地寻址一个字节、一个16位的字或一个32位的字，内存必须分成4个体。而4个体选择信号$\overline{BE0}$到$\overline{BE3}$分别用于选择一个体。很显然，每个体选择信号对应一个字节。当读写一个8位的字节时，4个体选择信号只有一个有效，选中1个体的8位数据。当读写一个16位的字时，4个体选择信号中有两个有效，同时选中2个体的16位数据。当读写一个32位的字时，4个体选择信号均有效，同时选中4个体共32位数据。
- 32位的数据信号 $D_0 \sim D_{31}$，这32位数据分为4个字节，分别是 $D_0 \sim D_7$、$D_8 \sim D_{15}$、$D_{16} \sim D_{23}$ 和 $D_{24} \sim D_{31}$。
- 控制信号 M/$\overline{IO}$(内存/接口选择信号，与8086一样)；D/$\overline{C}$(数据/控制)信号，低电平为处理器中止或正在响应中断，高电平表示正在传送数据；W/$\overline{R}$(读/写)信号，低电平表示读内存或接口，高电平表示写内存或接口。在486中，它们的功能如表7-2所示。

表7-2 控制信号编码表示的总线周期

M/$\overline{IO}$	D/$\overline{C}$	W/$\overline{R}$	总线周期
0	0	0	中断响应
0	0	1	停机
0	1	0	I/O读
0	1	1	I/O写
1	0	0	取指令操作码
1	0	1	保留
1	1	0	存储器读
1	1	1	存储器写

(b) 内存的连接

前面已经说明，在这里还需要再次强调，在32位的80386、80486系统中，内存必须分成为4个体。这与前面所描述的8位处理器8088不一样，在8位机中内存是不分体的或者说就是一个体。16位机分为两个体。32位机分成4个体。后面的Pentium机内存将分为8个体。

80386、80486系统中内存分体组织如图7-11所示。

在80x86中，内存都是按字节编址。因此，32位的数据要占4个内存地址。由图7-11可以看到，一个32位的数据分别放在4个不同的体中。

在80486系统中，利用4片容量为128K×8b的SRAM芯片构成512KB的内存，画出连接图如图7-12所示。

在图7-12中，4片SRAM构成4个存储体，分别利用$\overline{BE0} \sim \overline{BE3}$参加译码选中一个体。显然，在图7-12中，还可以有其他的译码形式，只要逻辑上是正确的，采用什么形式的译码电路并不很重要。

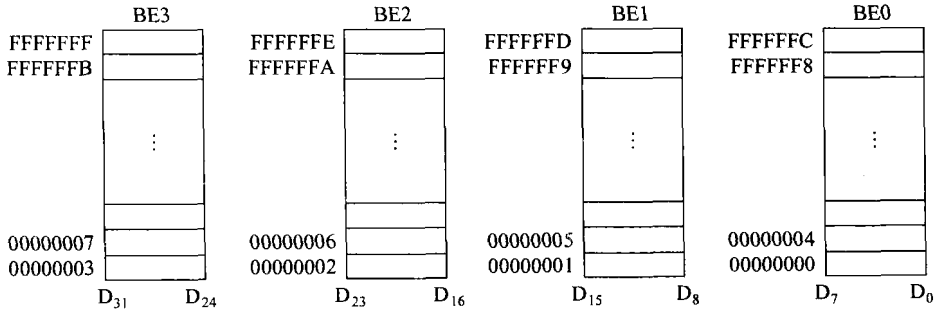


图 7-11　32 位机的内存组织

c) Pentium 处理器的内存组织

Pentium 处理器有几百条引线，其中与访问内存有关的是地址信号 $A_3 \sim A_{31}$、体选择信号 $\overline{BE0} \sim \overline{BE7}$、64 位数据信号 $D_0 \sim D_{63}$、及控制信号 $M/\overline{IO}$、$D/\overline{C}$ 和 $W/\overline{R}$。

从上面的信号可以注意到，Pentium 处理器对外部的数据线是 64 位的，可用 8 个字节来表示。同样，为了增加访问内存(或接口)的灵活性，须将内存分成 8 个体。为了方便对 8 个存储体的寻址，在处理器内部已将地址信号 $A_0 \sim A_2$ 进行译码，形成了 8 个存储体的选择信号 $\overline{BE0} \sim \overline{BE7}$。这与上面所说明的 80386、80486 的情况非常类似，所不同的是在 Pentium 处理器系统中内存要由 8 个体来构成，每个体对应一个体选择信号。具体内存芯片的连接这里不再说明。

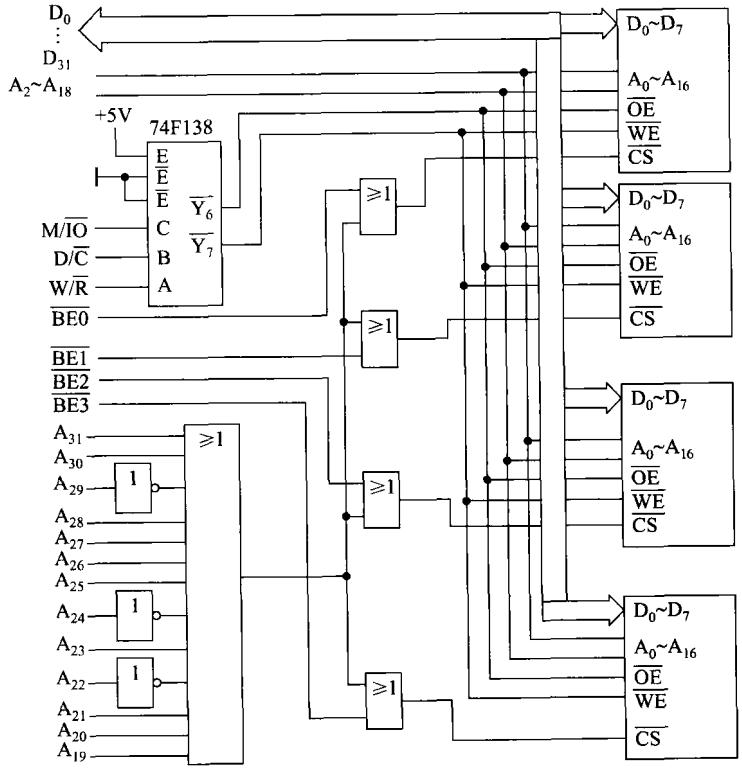


图 7-12　80486 内存芯片连接图

再次说明的是，上面构成内存的连接均是以80x86处理器的总线信号为例，存储器芯片也是特定的芯片。在实际的应用中，不同的处理器，不同的存储器芯片，它们的信号会略有不同。例如，在内存与接口统一编址（见后面章节说明）的处理器中，内存与接口的读写信号可以是$\overline{RD}$和$\overline{WR}$（或R/$\overline{W}$）；在内存与接口独立编址的处理器中，内存与接口的读写信号可以是$\overline{RD}$和$\overline{WR}$再加上M/$\overline{IO}$（或R/$\overline{W}$再加上M/$\overline{IO}$）；而存储器芯片上的$\overline{WE}$、$\overline{OE}$有可能只用R/$\overline{W}$来代替。

尽管有这些不同，但在构成内存的连接中，只要利用门电路就很容易实现信号间的匹配，详细情况不再说明。

7.2.2 只读存储器ROM

1. 概述

只读存储器ROM的重要特性是其存储信息的非易失性，存放在ROM中的信息不会因去掉供电电源而丢失，当再次加电时，其存储的信息依然存在。

（1）掩膜工艺ROM

这种ROM是芯片制造厂根据ROM要存储的信息，设计固定的半导体掩膜版进行生产的。一旦制出成品之后，其存储的信息即可读出使用，但不能改变。这种ROM常用于批量生产，生产成本比较低。计算机中一些固定不变的程序或数据常采用这种ROM存储。

（2）可一次编程ROM（PROM或OTP）

为了使用户能够根据自己的需要来写ROM，厂家生产了一种PROM（或一次可编程OTP）。允许用户对其进行一次编程——写入数据或程序。一旦编程之后，信息就永久性地固定下来。用户只可以读出和使用，但再也无法改变其内容。

（3）可擦去重写的PROM

这种可擦去重写的PROM是目前使用最广泛的ROM。这种芯片允许将其存储的内容利用物理的方法（通常是紫外线）或电的方法（通常是加上一定的电压）擦去。擦去后可以重新对其进行编程，写入新的内容。擦去和重新编程可以多次进行。一旦写入新的内容，就又可以长期保存下来（一般均在10年以上），不会因断电而消失。

利用物理方法（紫外线）可擦去的PROM通常用称为EPROM；用电的方法可擦除的PROM称为EEPROM（E^2PROM或EAROM）。这些芯片集成度高、价格低、使用方便，尤其适合科研工作的需要。

2. EPROM

EPROM是一种可以擦去重写的只读存储器。通常用紫外线对其窗口进行照射，即可把它所存储的内容擦去。之后，又可以用电的方法对其重新编程，写入新的内容。一旦写入，其存储的内容可以长期（几十年）保存，即使去掉电源电压，也不会影响到它所存储的内容。下面以一种典型的EPROM芯片为例来做介绍。

（1）EPROM 2764的引线

2764是一块8K×8b的EPROM芯片，其引线图如图7-13所示。

A_0～A_{12}为13条地址信号输入线，说明芯片的容量为8K个单元。

D_0～D_7为8条数据线，表明芯片的每个存储单元存放一个字节（8位二进制数）。在其

工作过程中，D_0～D_7 为数据输出线；当对芯片编程时，由此 8 条线输入要编程的数据。

$\overline{CE}$为输入信号。当它有效(低电平)时，能选中该芯片，故$\overline{CE}$又称为片选信号(或允许芯片工作信号)。

$\overline{OE}$是输出允许信号，当$\overline{OE}$为低时，芯片中的数据可由 D_0～D_7 输出。

$\overline{PGM}$为编程脉冲输入端。当对 EPROM 编程时，由此加入编程脉冲；读数据时$\overline{PGM}$为 1。

(2) EPROM 2764 的连接

2764 在使用时，仅用于将其存储的内容读出。其过程与 RAM 的读出十分类似，即送出要读出的地址，然后使$\overline{CE}$和$\overline{OE}$均有效(低电平)，则在芯片的 D_0～D_7 上就可以输出要读出的数据。

EPROM 2764 芯片与 8088 总线的连接图如图 7-14 所示。从图中可以看到，该芯片的地址范围在 F0000H～F1FFFH 之间。

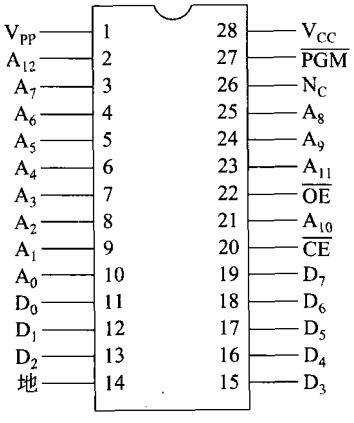


图 7-13 EPROM 2764 引线图

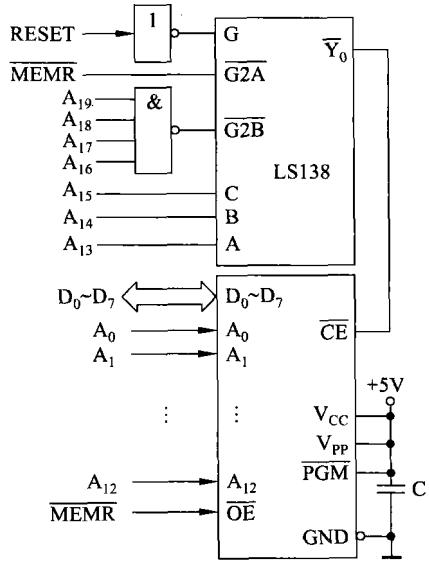


图 7-14 EPROM 2764 的连接图

(3) EPROM 的编程

EPROM 的一个重要优点就是可擦除重写，而且对其某一个存储单元来说，允许擦除重写的次数超过万次。

① 擦除

那些刚出厂未使用过的 EPROM 芯片均是干净的，干净的标志就是芯片中所有单元的内容均为 FFH。

若 EPROM 芯片已使用过，则在对其编程前必须将其从系统中取下来，放在专门的擦除器上进行擦除。擦除器利用紫外线照射 EPROM 的窗口，一般 15～20min 即可擦除干净。

② 编程

对 EPROM 的编程通常有两种方式，即标准编程和快速编程。

a) 标准编程

标准编程过程为：将 EPROM 插到专门的编程器上，先使 Vcc 引脚加到＋5V，然后 Vpp 加上 EPROM 芯片所要求的高电压(如＋12.5V，＋15V，＋21V，＋25V 等)，然后在地址线上加上要编程单元的地址，数据线上加上要写入的数据。$\overline{CE}$使保持低电平，$\overline{OE}$为高电平；上述信号全部达到稳定后，在$\overline{PGM}$端加上(50±5)ms 的负脉冲，这样就将一个字节的数据写到了相应的地址单元中。重复上述过程，即可将要写入的数据逐一写入相应的存储单元中。

每写入一个地址单元，在其他信号不变的条件下，将$\overline{OE}$变低，可以立即读出校验；也可在所有单元均写完后再进行最终校验；还可采用上述两种方法进行校验。若写入数据有错，则可从擦除开始，重复上述过程再进行一次写入编程过程。

标准编程用在早期的 EPROM 中。这种方式编程有两个主要的缺点：其一是编程时间太长。当 EPROM 容量很大时，每个单元 50ms 的编程时间，使写一块大容量芯片时间长得令人无法接受；其二是不够安全。编程脉冲太宽致使功耗过大而损坏 EPROM。

b) 快速编程

随着技术的进步，EPROM 芯片的容量越来越大。同时，也研制出相应的快速编程方法。编程时，其他步骤与标准编程都一样，不同的仅仅是使用编程脉冲已变窄，一般为 0.1ms 到 2ms。例如，27C040 所要求的编程脉冲的宽度仅为 100μs。因此，其编程时间会大大缩短。

3. EEPROM

EEPROM 就是电擦除可编程只读存储器的英文缩写。EEPROM 在擦除及编程上比 EPROM 更加方便。EEPROM 的出现将使得 EPROM 逐渐地退出历史舞台。

(1) 典型 EEPROM 芯片介绍

EEPROM 以其制造工艺及芯片容量的不同而有多种型号。下面仅以 8K × 8b 的 EEPROM NMC98C64A 为例来加以说明。其引线如图 7-15 所示。

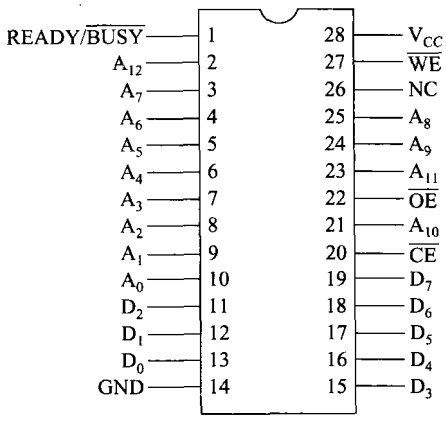


图 7-15 EEPROM NMC 98C64A 引线图

A_0～A_{12}为地址线，用于选择片内的 8K 的存储单元。

D_0～D_7 为 8 条数据线，表明每个存储单元存储的一个字节的信息。

$\overline{CE}$为片选信号。当$\overline{CE}$为低电平时，选中该芯片；当它为高电平时，该芯片不被选中。芯片未被选中时，芯片的功耗很小，仅为 $\overline{CE}$ 有效时的 1/1000。

$\overline{OE}$为输出允许信号。当$\overline{CE}$=0，$\overline{OE}$=0，$\overline{WE}$=1 时，可将选中的地址单元的数据读出。

$\overline{WE}$是写允许信号。当$\overline{CE}$=0，$\overline{OE}$=1，$\overline{WE}$=0 时，可以将数据写入指定的存储单元。

READY/$\overline{BUSY}$是漏极开路输出端，当写入数据时该信号变低，数据写完后，该信号变高。

(2) 98C64A 的工作过程

EEPROM 98C64A 工作过程如下所述。

① 读出数据。由 EEPROM 读出数据的过程与从 EPROM 及 RAM 中读出数据的过程是一样的。当$\overline{CE}=0$,$\overline{OE}=0$,$\overline{WE}=1$ 时,只要满足芯片所要求的读出时序关系,则可从选中的存储单元中将数据读出。

② 写入数据。将数据写入 EEPROM 98C64A 有两种方式。

第一种是按字节编程方式,即一次写入一个字节的数据,过程是:使$\overline{CE}=0$,$\overline{OE}=1$,地址线上加上要写的地址,数据线上加上要写入的数据。在$\overline{WE}$端加上 100ns 的负脉冲,便可以启动数据的写入。这里要特别注意的是,$\overline{WE}$脉冲过后 READY/$\overline{BUSY}$端为低电平,表示写入过程正在进行,直到 READY/$\overline{BUSY}$端的低电平变高,才完成一个字节的写入。这段时间里包括了对本单元数据擦除和新数据写入的时间。不同芯片所用时间略有不同,一般是几百微秒到几十毫秒,98C64A 需要的 T_{WR}为 5ms,最大为 10ms。

在对 EEPROM 编程过程中,可以通过程序查询 READY/$\overline{BUSY}$信号或利用它产生中断来判断一个字节的写入是否已经完成。

可以看到,在对 EEPROM 编程时,可以在线操作,即可在计算机系统中直接进行,从而减少了不少麻烦。

第二种编程方法称为自动按页写入。在 98C64A 中一页数据最多可达 32 个字节,要求这 32 个字节在内存中是顺序排列的。

页编程的过程是,连续写入一页,用查询或中断看 READY/$\overline{BUSY}$是否已变高,若变高,则写周期完成。接着可以写下一页,直到将数据全部写完。

新的 EEPROM 一页可达数 KB,连续写满一页仅等待几 ms,其编程速度很快。

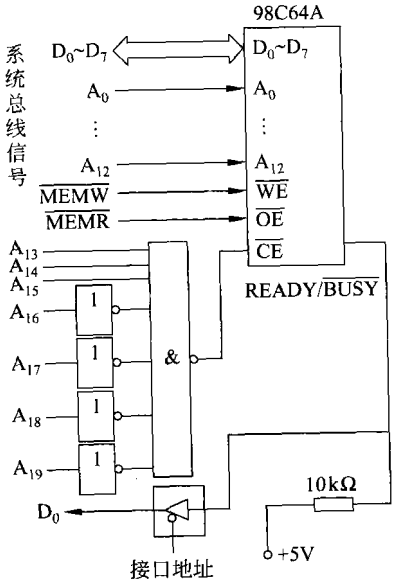


图 7-16 EEPROM 98C64A 的连接

(3) 连接使用

EEPROM 可以很方便地接到计算机系统中。图 7-16 就是将 98C64A 连接到 8088 总线上的连接图。

除了上面提到的并行 EEPROM 之外,尚有多种型号的串行 EEPROM。它们都是在外接时钟驱动下一位接一位的写入或读出。因此,它们的容量一般都比较小,几百 b 到几 Kb,读写速度都很慢。多用于银行卡、各种预付费卡等。

(4) 闪速存储器

闪速(flash)存储器也是一种电可擦除可多次编程的存储器。与前面描述的 EEPROM 不同之处在于其擦除写入的速度比较快,为强调其写入速度快才称为闪速存储器。闪速存储器在制造工艺主要有两类:或非(NOR)型阵列和与非型(NAND)阵列。

NOR 的传输效率很高,在 1~4MB 的小容量时具有很高的成本效益。

NAND 结构能提供极高的单元密度,可以达到高存储密度,并且写入和擦除的速度也

很快。当选择存储解决方案时，必须权衡以下的各项因素。

- NOR 的读速度比 NAND 稍快一些。
- NAND 的写入速度比 NOR 快很多。
- NAND 的 4ms 擦除速度远比 NOR 的 5s 快。
- 大多数写入操作需要先进行擦除操作。

使用时应注意接口上的差别，NOR flash 带有 SRAM 接口，有足够的地址引脚来寻址，可以很容易地并行存取其内部的每一个字节。因此，它们可以直接接系统总线，构成内存储器。

NAND flash 器件使用复杂的 I/O 口来串行地存取数据，采用串行接口，不能直接构成内存，只能用它们构成外存储器。很显然，基于 NAND 的存储器就可以取代硬盘或其他外存储设备。

7.2.3 动态读写存储器 DRAM

在上一节中已介绍过动态存储器的单元电路，利用这样的单元电路构成的动态读写存储器速度快、集成度高、功耗小、价格低，在计算机中得到极其广泛的使用。PC 中的内存条无一例外地采用动态存储器，现在的许多嵌入式计算机系统中也越来越多地使用这种存储器。

1. 概述

目前，大容量的 DRAM 芯片已研制出来，为构成大容量的存储器系统提供了便利的条件。下面以一种简单 DRAM 芯片来例说明 DRAM 的工作原理。

(1) 动态存储器芯片 2164 的引线

2164 是一块 64K×1b 的 DRAM 芯片。这种存储器芯片引线与 SRAM 有所不同，请读者注意。2164 的引线如图 7-17 所示。

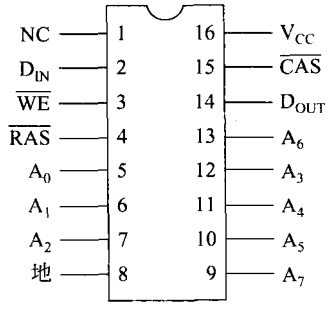


图 7-17 DRAM 2164 引线图

$A_0 \sim A_7$ 为地址输入端。在 DRAM 芯片的构造上，芯片上的地址引线是复用的。可以看到，2164 的容量为 64K 个单元，每个单元存储一位二进制数。如何来寻址这 64K 个单元呢？在存取芯片的某单元时，其操作过程是将存取的地址分两次输入到芯片中去，每一次都是由 $A_0 \sim A_7$ 输入的。两次加到芯片上去的地址分别称为行地址和列地址。

在芯片内部，各存储单元是一种矩阵结构排列。行地址在片内译码选择一行，列地址在片内译码选择一列。这样由选中的行和列来决定所选中的单元，可以简单地认为该芯片有 256 行和 256 列，共同决定 64K 个单元。对于其他 DRAM 芯片也可以同样来考虑。

综上所述，动态存储器芯片上的地址引线是复用的，CPU 对它寻址时的地址信号分在行地址和列地址上，分别由芯片上的地址线送入芯片内部进行锁存、译码而选中要寻址的单元。

D_{IN}和D_{OUT}是芯片上的数据线。其中D_{IN}为数据输入线，当CPU写芯片的某一单元时，要写入的数据由D_{IN}送到芯片内部。同样，D_{OUT}是数据输出线，当CPU读芯片的某一单元时，数据由此引线输出。

$\overline{RAS}$为行地址锁存信号。利用该信号将行地址锁存在芯片内部的行地址缓冲寄存器中。

$\overline{CAS}$为列地址锁存信号。利用该信号将列地址锁存在芯片内部的列地址缓冲寄存器中。

$\overline{WE}$为写允许信号。当该信号为低电平时，允许将数据写入。反之，当$\overline{WE}=1$时，可以从芯片读出数据。

(2) DRAM的工作过程

① 读出数据

当要从DRAM芯片读出数据时，CPU首先将行地址加在$A_0 \sim A_7$上，然后送出$\overline{RAS}$锁存信号，该信号的下降沿将行地址锁存在芯片内部。接着将列地址加到芯片的$A_0 \sim A_7$上，再送$\overline{CAS}$锁存信号，该信号的下降沿将列地址锁存在芯片内部。然后保持$\overline{WE}=1$，则在$\overline{CAS}$有效期间(低电平)数据输出并保持。其过程如图7-18所示。

② 写入数据

当需要将数据写入芯片时，前面的过程即锁存地址的过程与读出数据时一样，行列地址先后由$\overline{RAS}$和$\overline{CAS}$锁存在芯片内部。然后$\overline{WE}$有效(为低电平)，加上要写入的数据，则将该数据写入选中的存储单元，如图7-19所示。

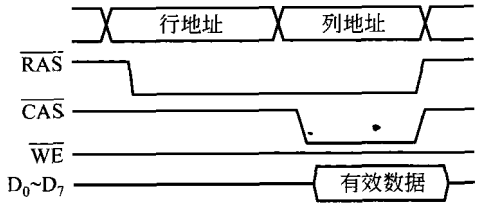


图7-18 DRAM 2164的读出过程

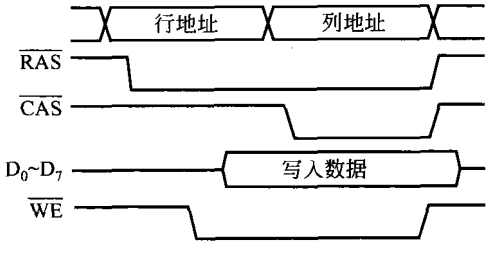


图7-19 DRAM 2164的写入过程

图7-19中，$\overline{WE}$变为低电平是在$\overline{CAS}$有效之前，通常称为提前写。这能够将输入端D_{IN}的数据写入，而D_{OUT}保持高阻状态。若$\overline{WE}$有效(低电平)出现在$\overline{CAS}$有效之后，且满足芯片所要求的滞后时间，则$\overline{WE}$开始是处于读状态，然后才变为写状态。这种情况下，能够先从选中的单元读出数据，出现在D_{OUT}上。然后，再将D_{IN}上的数据写入该单元。这种情况可一次同时完成读和写，故称为读变写操作周期。

③ 刷新

动态RAM一个重要的问题是，它所存储的信息必须定期进行刷新。对于单元电路的刷新过程上节已经说明。通常DRAM要求每隔2～4ms刷新一次。

动态存储器芯片的刷新过程是每次送出行地址加到芯片上去，利用$\overline{RAS}$有效将行地址锁存于芯片内部，这时$\overline{CAS}$保持无效(高电平)。这样就可以对这一行的所有列单元进行刷新。每次送出不同的行地址，顺序进行，则可以刷新所有行的存储单元。也就是说行地址循环一遍，则可将整个芯片的所有地址单元刷新一遍。只要保证在芯片所要求的刷新时间内

(2～4ms)刷新一遍，也就达到了定期刷新的目的。刷新波形如图 7-20 所示。

在刷新中，$\overline{CAS}$保持无效，只利用$\overline{RAS}$锁存刷新的行地址，进行逐行刷新。即图 7-4 中所描述的 Y 无效 X 有效，每次刷新一行。

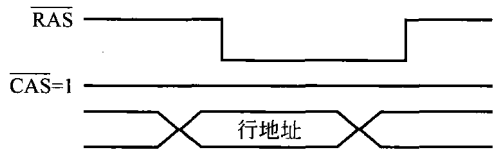


图 7-20 DRAM 2164 的刷新过程

2. 内存条

内存条是 PC 的重要组成部分。它是将多片存储器芯片焊在一小条印刷电路板上，构成容量不一的小条。使用时将小条插在主板的内存条插座上。由于 PC 的内存要求容量大、速度快、功耗低且造价低廉，而动态存储器恰恰具备这些性能。因此，PC 的内存条无一例外地都采用动态存储器。

随着 PC 的发展，构成内存条的 DRAM 也经历了若干代的变更。早期的 PC 所用的 PM DRAM、EDO DRAM 已经不再使用，这里不做说明。当前还在用的芯片主要是 SDRAM 和 DDR SDRAM。

目前，PC 上的内存条主要是由 SDR SDRAM(单倍速率同步 DRAM)或 DDR SDRAM 芯片构成。

标准的 DDR 内存条是 184 引脚线 DIMM(双面引脚内存条)。它很像标准的 168 引脚线 SDRAM DIMM，只是用了一个凹槽而不是 SDR 上的 2 个凹槽。组件的长度也是 5.25 英寸。

标准化协会定义了两种不同配置的 DDR 内存条。第一种是无缓冲 DDR DIMM，它成本低，可应用在 PC 和 Internet 设备上。第二种是有缓冲 DDR DIMM，应用于较高存储密度的服务器。

新近的 DDR-Ⅱ内存条所用的 DDR 芯片的速度更高一些。目前有三种工作频率：400MHz、533MHz 和 667MHz，分别可以达到的速率为 7.8GBps、5.6GBps 和 6.4GBps。所有的 DDR-Ⅱ内存条均工作在 1.8V 电压之下，单条容量均在 512MB 以上。DDR-Ⅱ内存条的引脚线有 200 线、220 线和 240 线几种。

7.2.4 其他存储器

除上面所提到的存储器外，尚有多种其他用途的存储器。这里仅介绍如下几种。

1. 多端口存储器

在计算机系统中，经常利用多端口存储器实现处理机之间的快速数据交换。多端口存储器包括双端口、三端口、4 端口等多种。这里仅对双端口存储器做简要介绍。

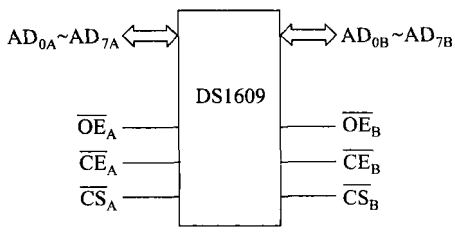


图 7-21 双端口存储器 DS1609 的引线

(1) 双端口存储器

双端口存储器有多个厂家生产的多种型号。在这里仅以一种为例说明它的应用。

① 引线

双端口存储器 DS1609 的引线如图 7-21 所示。

由图 7-21 可以看到，双端口存储器的引线分为两个独立的端口，分别在图 7-21 的两侧。

引线 $AD_0 \sim AD_7$ 为复用引线，这 8 条线上可输入地址信号，也可以传送数据。其他控制信号均已经熟悉，不再说明。

② 同时操作

双端口存储器就存在 A、B 两端口对其存储单元同时操作，下面分别说明。

a) 对不同存储单元允许同时读或写。

b) 允许对同一单元同时读。

c) 当一个端口写某单元而另一端口同时读该单元时，要么是读出旧数据，要么是写入新的数据。因此，这种情况也不会发生混乱。

d) 当两个端口同时对同一单元写数据时，就会引起竞争，产生错误。因此，这种情况应想办法加以避免。

③ 竞争的消除

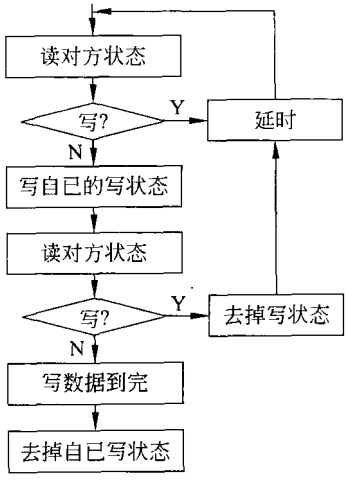


图 7-22 某一边的写过程

④ 对于 DS1609 来说，竞争发生在对同一单元同时写数据时。为了防止竞争的发生，可以另外设置两个接口，该接口能保证一个端口只写而另一个只读。例如，用带有三态门输出的锁存器来实现，74LS373、74LS374 均可实现。如果可能，也可在 DS1609 中设置两个单元一个 A 端口只写而 B 端口是只读，另一个单元则相反，B 端口只写而 A 端口只读。在 A 端口向 DS1609 写数据时，先读 B 端口的写状态。若 B 端口不写，则将自己的写状态写到存储单元中。当 B 端口写入时，同样需要查询 A 端口的状态。其过程可用图 7-22 所示的流程图来说明。

不同的双端口存储器在结构上有所不同，上述芯片没有状态信号，所需状态信号由使用者产生。而有的双端口存储器芯片有自己的状态信号，如 BUSY，使用者在使用过程中可用芯片所产生的这种信号。

2. 多体交叉存储器

(1) 多体并行访问

多体交叉存储器在本章的前一节中已经涉及，只是没有特别强调。例如，8086 的内存连接中采用两个体——奇地址体及偶地址体的交叉编址方式。而 80386 和 80486 的内存则采用 4 体交叉编址方式。参见图 7-11 可以看到 4 个体的地址编排不是顺序的而是交叉的。同样的思路 Pentium 的内存是 8 体交叉编址。

在上面 80x86 系列处理器中，处理器对内存的存取是以增加数据总线的宽度达到提高内存访问速度的目的的。在 8086 中，可一次读(或写)16 位数据。若数据总线只有 8 位的话，处理器要两次才能读写 16 位。显然，速度提高了 1 倍。同样，386、486 数据总线宽度增加到 32 位，比 8 位总线的速度提高了 4 倍；而 Pentium 处理器的数据总线宽度增加到 64 位，则比 8 位总线的速度提高了 8 倍。

在 80x86 系列中，处理器对内存的访问多个存储体同时进行，或者说是多体并行访问。这是多体交叉存储器的一种访问方式。这种方式实现比较容易(见前面的连接图)，而且速

度很快。但需要更宽的数据总线宽度。

为提高内存的访问速度，多体交叉存储器还有另一种访问方式，就是多个模块顺序轮流进行访问，也称为交叉访问。

(2) 多体交叉访问

如前所述，多体存储器中的体的个数一般是 2 的幂次方，如 2、4、8 个或更多。为了说明交叉访问，将 4 个体的多体存储器简化框图如图 7-23 所示。

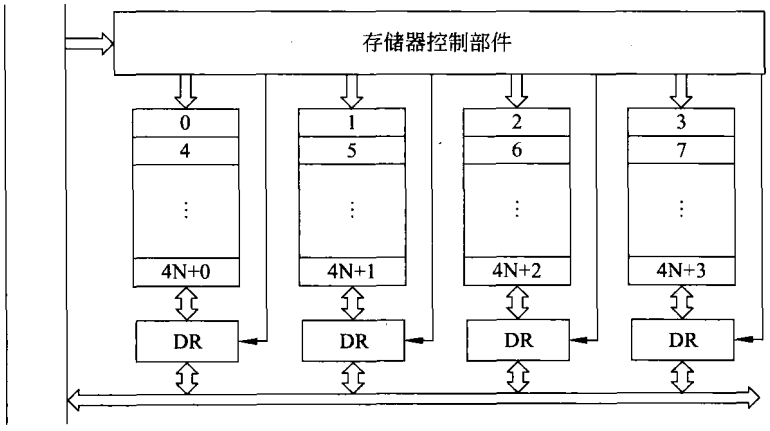


图 7-23 4 体交叉访问存储器简化框图

由图 7-23 可以看到，存储器地址与 80x86 一样交叉编址。4 个存储体是完全独立的，各自有自己的地址寄存器、地址译码器和数据寄存器 DR。而且数据总线的宽度仅为每个存储单元的数据宽度。

该存储器的访问过程是交叉进行的，为此利用如图 7-24 的示意图来加以说明。

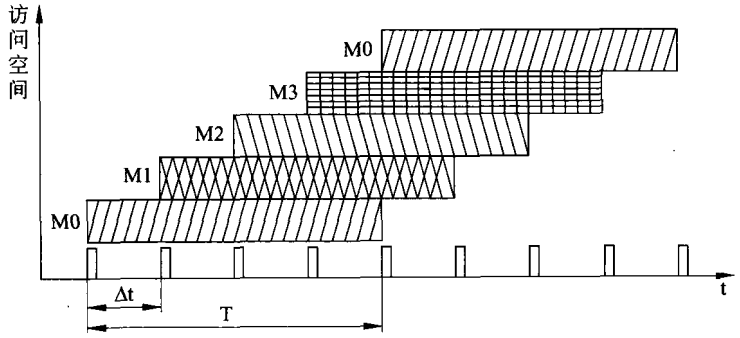


图 7-24 4 体存储器交叉访问示意图

在图 7-24 中，假定每个存储体的访问时间为 T。显然，顺序访问任何一个体的 4 个连续地址则需要 4T。同时假定控制部件每隔 Δt 启动一个存储模块。并且在图 7-24 的情况下保证 $4\Delta t \leqslant T$，图中采用 $4\Delta t = T$ 的情况。也就是说应保证 $T/\Delta t$ 应大于等于存储体的数量。只有这样才能保证连续访问时存取数据的正确性。

在对 4 个体进行一次访问时，访问命令启动存储器控制部件产生 4 个等间隔控制脉冲信号，顺序启动对 4 个体 M0、M1、M2 和 M3 的访问。在读出数据时，经 T 时刻都将数据读

到 DR 中。T 时刻开始 CPU 只需再用 3 个 Δt 就可以将数据读走。

可以看到，顺序访问任何一个体的 4 个连续地址则需要 $4T=16\Delta t$，而采用交叉访问的情况下只需要 $7\Delta t$。

当连续读出 m 个数据时，由图 7-24 可知，在读出前面数据的同时，又可以启动后续数据的读出。这样就形成一种类似流水线方式的工作，当等待 M0 体读出后(用 T 时间)，则每个 Δt 可读出一个数据。因此，连续读出 m 数据时，只需 $T+(m-1)\Delta t$。当 m 很大时可近似读出一个数据的时间就是 Δt。显然，速度就提高了许多。

在上面的说明中，是以交叉读出为例来说明的，交叉写入数据的思路是一样的。在连续写大量数据时，可以认为 CPU 是以 Δt 的速度交叉写每个存储体单元，而存储体单元的写入时间依旧是 T。速度就提高了。

从以上分析可见，采用多体交叉存储器可以有效地提高存储器的存取速度。但多体交叉访问的形式已成为历史。

3. 相联存储器

前面所描述的存储器都是按地址进行访问的，也就是说这些存储器都是依照地址去寻找地址中的内容。而相联存储器是依据内容决定内容的地址或者是依据内容去寻找与其相关的内容。

相联存储器的构成框图如图 7-25 所示。其组成主要包括如下部件。

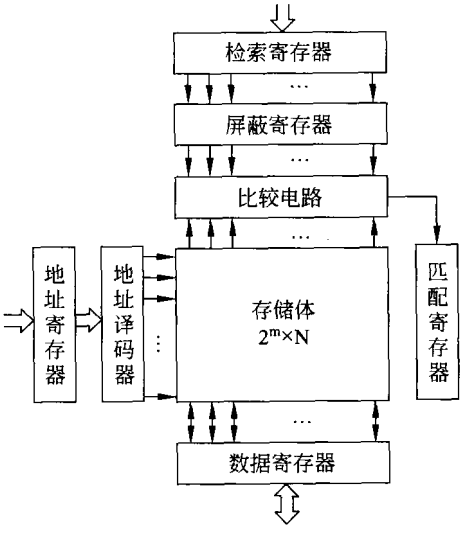


图 7-25　相联存储器的构成图

(1) 检索寄存器

用于存放要检索的内容，其长度为 N 位二进制编码。

(2) 屏蔽寄存器

屏蔽寄存器用于将不进行检索的某些位屏蔽掉，只留下需要检索的各位参与比较。通常屏蔽寄存器中某位置 1，则将相应的检索位屏蔽掉；若置 0 则保留检索位。

(3) 存储体

用于存放信息的部件，假定存储体的容量为 $2^m \times N$，即它具有 2^m 个存储单元，每个存储单元存储 N 位二进制信息。

(4) 比较电路

比较电路是由 2^m 个 N 位的数字比较器构成，用来将存储体中所有存储单元的内容与经屏蔽后的检索字进行比较。采用如此数量的硬件比较器，进行并行的同时比较，为的就是提高比较器的速度。

(5) 匹配寄存器

匹配寄存器用于记录比较器的比较结果。该寄存器有 2^m 位，刚好对应存储体的 2^m 个存储单元。当某一存储单元存储的信息与屏蔽后的检索字相同(匹配)时，该位置 1；未匹配存储单元的相对应位则为 0。

(6) 数据寄存器

数据寄存器用来存放比较过程中发现的匹配存储单元的内容。加电后,其内容为随机数。

如图 7-25 所示,还在左侧画出了地址寄存器和地址译码器,为的是使相联存储器具有按地址访问的功能。

可以想象,相联存储器中使用了数字比较器,用来比较经屏蔽后的检索字与存储单元的内容是否相等。数字比较器还可以进行大于、小于等比较功能。因此,有的相联存储器具有大于、小于等其他功能的检索。

相联存储器可用于许多领域,本书中主要用在高速缓冲存储器及虚拟存储器中。

7.3 高速缓冲存储器

在过去的 30 年里,CPU 速度的提高非常快而存储器速度增加相对比较慢,与 CPU 速度相匹配的速度高的存储器价格很贵。这样一来就产生了矛盾:若计算机内存全部采用低价的慢速存储器作为内存,则 CPU 的高速度无法发挥出来。若全部采用高价的快速存储器作为内存,CPU 的高速度是能够发挥出来了,但价格又使人无法接受。就 PC 而言,从 80386 CPU 之后,人们采用高速缓冲存储器(cache)来解决上面的矛盾。使计算机既具有低的价格又发挥出了 CPU 的高速度。

7.3.1 工作原理

1. 概述

(1) 高速缓冲存储器的位置

高速缓冲存储器的工作是建立在程序及数据访问的局部性原理之上。经过对大量程序执行情况的结果分析表明:在一段较短的时间间隔内程序集中在某一较小的内存地址空间执行,这就是程序执行的局部性原理。同样,对数据的访问也会存在局部性现象。

基于上述程序及数据访问的局部性原理,在 CPU 和主存之间,尽量靠近 CPU 的地方设置一种容量比较小而速度高的存储器,将当前正在执行的程序和正在访问的数据放在其中。在程序运行时,不需要从慢速的主存中取指令和数据,而是直接访问这种高速小容量的存储器,从而可以提高 CPU 的程序执行速度,这种存储器就称为高速缓冲存储器。

高速缓冲存储器所处的位置如图 7-26 所示。

(2) cache 的构成

cache 的构成框图如图 7-27 所示。主要包括替换管理模块、由相联存储器构成的存储表以及小容量高速度存储器。

当 CPU 执行程序需从主存读指令或数据时,从地址线上将地址加到主存也加到相联存储表上。若在相联存储表中检索到要读写的信息就在高速 cache 中,则通过相联存储表将地址转换为 cache 地址,从中读出信息。这种情况称为“命中”。若在相联存储表中未能

检索到要读写的信息，则称“未命中”。此时，CPU 用主存读周期从主存中读出指令或数据，同时将包括读出信息在内的一块主存的信息读出替换 cache 中的某一块。

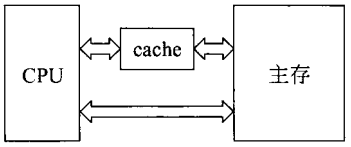


图 7-26　cache 所处位置

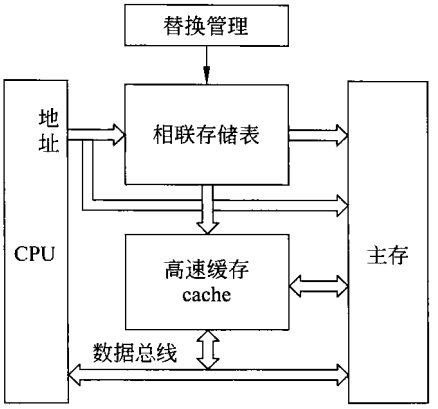


图 7-27　cache 的组成

值得注意的是，cache 中存储的内容与主存的相应部分的内容是完全一样的，是主存内容的复制。只是为了提高运行速度才将这部分内容放在 cache 中的。同时，为了提高速度，整个 cache 系统都是由硬件实现的。

由于程序执行的局部性原理，CPU 在绝大多数的情况下都可命中。因此，CPU 可以以更高的速度运行程序。有关 cache 工作的细节将在下面详细说明。

2. 地址映射

如上所述，在 cache 工作过程中，需要将主存的信息复制到 cache 中，这就是主存地址到 cache 地址的“地址映射”。在程序执行中，如果命中，就从 cache 中存取信息。这就需要将主存地址转换为 cache 的地址，才能对 cache 进行访问。将主存地址转换为 cache 地址称为“地址变换”。地址映射与变换有如下三种基本方式。

(1) 全相联方式

为了工作方便，提高效率，通常将 cache 和主存分成容量相等的块。例如，主存 64MB，cache 为 32KB，均将它们分为大小为 4KB 的块。在全相联方式下，cache 被分为块号为 0～7 的 8 块，块号可用 3 位二进制编码表示。而主存则分为 16K 块，块号从 0～16 383，块号需要用 14 位二进制编码表示。

全相联方式下，地址映射的法则是：主存的任何一块可从装入到 cache 的任何一块中。这时，存放主存块号的相联存储器应有 cache 块数个单元(上例为 8)。在相联存储器中需要记录装入 cache 的主存块的块号，因此，相联存储器每个单元应为 14 位。相联存储器的容量为 8×14b。全相联地址映射示意图如图 7-28 所示。

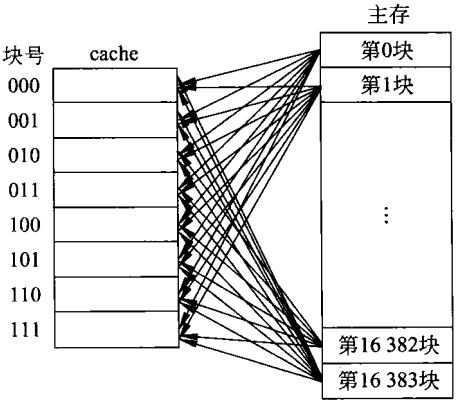


图 7-28　全相联地址映射

由图 7-28 可以看到，主存的每一块均可以装

入 cache 的任一块中，没有什么限制。

在全相联方式下，地址的变换过程如图 7-29 所示。

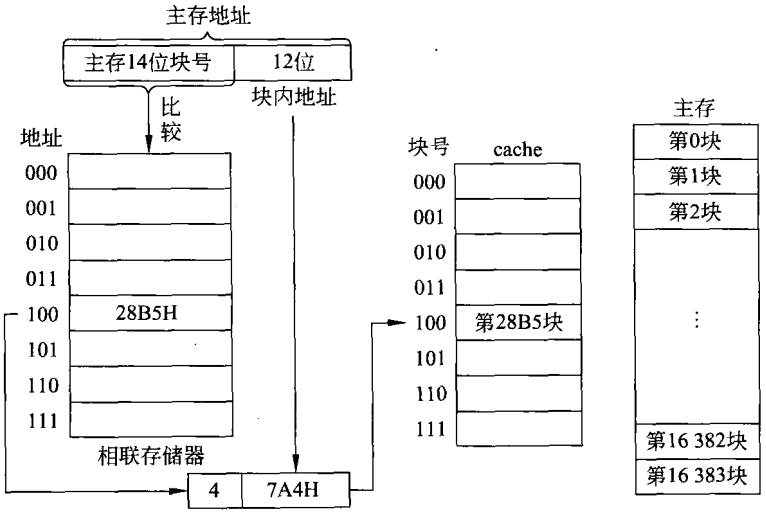


图 7-29 全相联地址变换

为了说明问题方便，主存及 cache 的大小及分块仍以上面所给出的特定数据。同时，图 7-29中画出了 8×14b 的相联存储器。它只有 8 个存储单元，用 3 位编码来表示存储单元的地址。而且，地址号与 cache 的块号是完全对应的。

假定某时刻主存第 28B5H 块(即从 28B5000H 到 28B5FFFH 这 4KB)已经装入 cache 的第 100B(4)块中。这时，该 cache 块的地址是 32KB 中的 4000H 到 4FFFH 这 4K 个地址。同时，要将块号 28B5H 存在对应的相联存储器地址为 100B(4)的单元中，如图 7-29 所示。

若此后某时刻，CPU 送出主存地址为 28B57A4H，其低 12 位为块内地址，而高 14 位 28B5H 为主存块号。利用相联存储器，在其中检索比较主存块号 28B5H，由内容 28B5H 寻找到其地址为 100B(4)。这就是 cache 的块号——高位地址。最后将此高位地址与块内地址连接在一起，便形成了 cache 地址——47A4H。CPU 从 cache 的 47A4H 单元即可访问到主存 28B57A4H 单元的内容。

全相联映射的优点是主存装入 cache 特别灵活，主存的任一块都可装入 cache 的任一块中。只有全部装满后才需要替换。在替换时，根据算法只需淘汰掉任何一块即可装入新的。

但是，这种方式的代价也是最高的。它要求的相联存储器容量最大，而且进行比较检索时需全部检索相联存储器才能获得 cache 的块号，地址变换机构最复杂。

(2) 直接映射方式

为了说明直接映射，下面仍以前面所给出的数值为例。在直接映射方式下，要将主存先以 cache 的容量 32KB 分区。则 64MB 的主存就被分成 0～2047 个区——即 2K 个区。而后每个区再以 4KB 分块，如图 7-30 所示。

利用图 7-30，说明地址直接映射的法则为：主存各区中块号相同的块只能装入与 cache 中相同的块号中。例如，所有区中的 0 号块只准装入 cache 的 0 号块中；1 块号只能装入

cache 的 1 块号中，以此类推。

由上所述可以看到，这种映射使主存块号与 cache 块号永远是一致的。

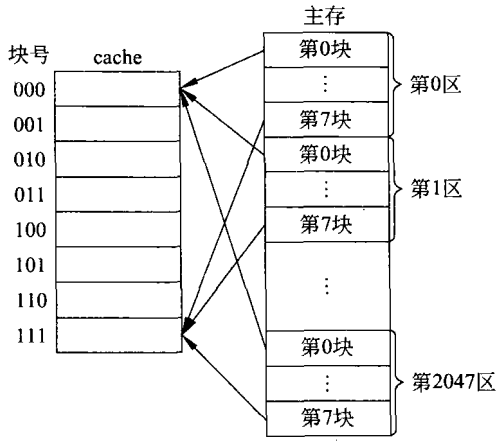


图 7-30　直接映射方式示意图

下面，分析直接映射的地址变换过程。同样假定某一时刻 CPU 送出主存地址为 28B57A4H。此时该主存地址应由如下部分组成，如图 7-31 所示。

由图 7-31 可以看到，地址的低 12 位是块内的地址；中间 3 位用于表示区内的块号。这里要特别注意的是直接映射方式下，该 3 位表示的也一定是 cache 的块号。因为主存的 101B(5)块一定要装入 cache 的 101B(5)块中。故假如命中的话，这 3 位连同后面的 12 位块内地址就是 cache 的地址 57A4H。地址的最高 11 位用来表示区号，即区号为 516H。

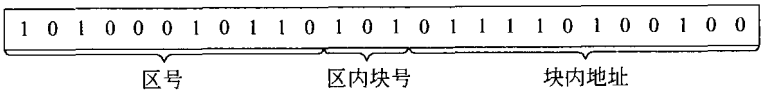


图 7-31　直接映射方式下主存地址分析

下面看一看地址的变换过程。假如在此之前 CPU 已访问过区号为 516H、区内块号为 101B(5)的块。则该块一定已经装入 cache 的 101B(5)块中。同时，在相应的地址是 101B 的相联存储单元中必须存储主存的区号 516H，如图 7-32 所示。

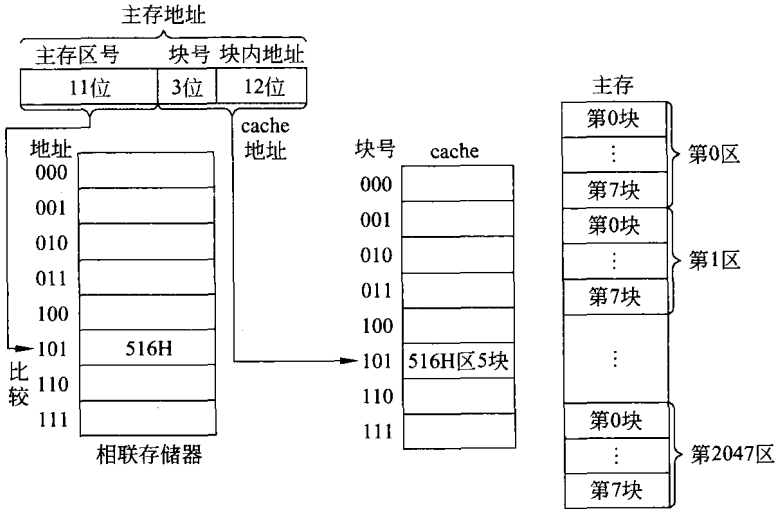


图 7-32　直接地址变换

利用图 7-32 所示的直接方式下的地址变换过程，当 CPU 送出主存地址为 28B57A4H 时，直接映射的变换机制得到此时的区号为 516H，而块号为 5(101B)，则直截了当只检索比较相联存储器的 5(101B)号单元的内容是否是 516H。若是，则为命中，主存地址的低 15 位

就是 cache 的地址，则地址就是 57A4H。若不是，则没有命中。

可见，直接映射方式下的地址变换变得很简单，只需根据主存地址中得到的块号——即相联存储器地址去检索该单元存放的是否定本块地址的区号。是则命中；不是则未命中。

综上所述，直接映射方式地址变换特别简单，只需以主存地址的块号为地址检测相联存储器（实际上，这种情况下已经是一种一般存储器）的一个存储单元。若命中，则由主存地址可直接获得 cache 的地址。该方式的缺点就是效率低，当某一块号正在使用而其他区同一块号又需装入时，即使其他块空着或很久都未使用也不能装入，只能将正在使用的替换。也就是说不同区的相同块号是不允许同时存在于 cache 中，若出现不同区的相同块号频繁交替被访问时，就会出现这些块被频繁装入的情况，这将大大影响命中率。因此，这种方式更适合于大容量 cache。

(3) 组相联方式

前面已经看到，全相联方式和直接映射方式各有优缺点。而组相联方式就是将上述两者的优点结合而尽可能减小其缺点。该方式的基本思想是将 cache 先分组，组内再分块；而主存是先以 cache 的总容量分区，区内按 cache 的方法分组、组内再分块。

地址映射是组间直接映射而组内的块全相联映射。也就是说主存的某号组只能装入 cache 的同号组中，而组内的块可以装入同组号的任意一块中。具体示意图如图 7-33 所示。

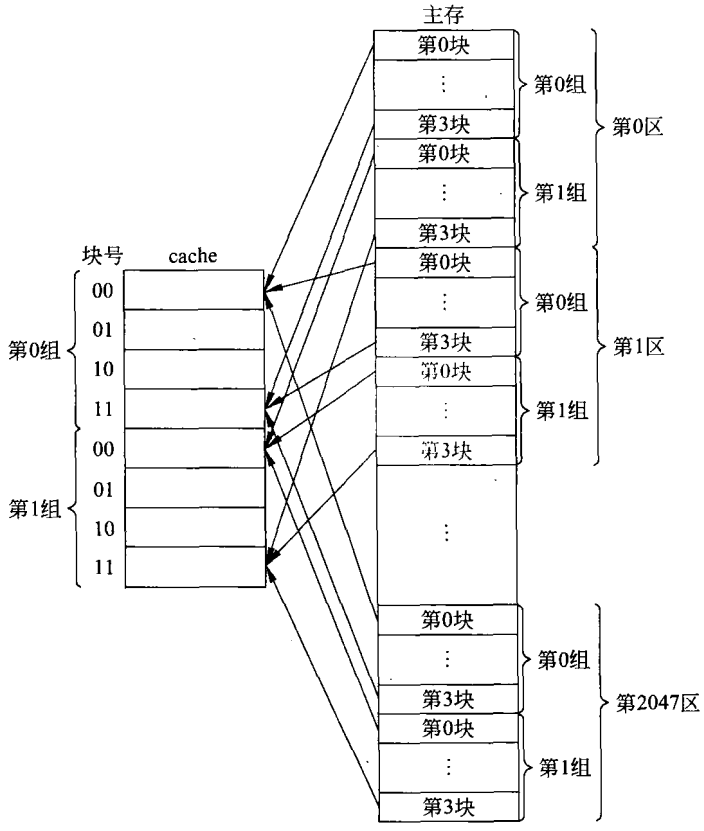


图 7-33 全相联地址映射示意图

在图 7-33 中，将 cache 分为 0、1 两组，每组分为 00～11 4 块。同样，将主存分区，每区分为 0、1 两组，每组分为 00～11 4 块。图 7-33 中表明各区中同号组只能调入 cache 的同号组中，而组内块是全相联的。

作为前面例中 CPU 给出的主存地址为 28B57A4H，在组相联地址变换中，对该地址分析如图 7-34 所示。

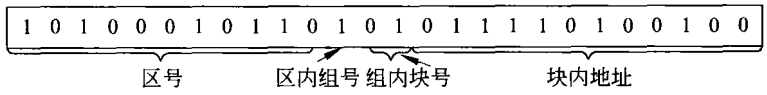


图 7-34　组相联方式下主存地址分析

由图 7-34 分析可知，设主存地址的区号仍为 516H；区内的组号为 1，这就是说该组内的所有块只能装入 cache 的第 1 组中；组内的块号为 01B(第 1 块)，它可以装入 cache 第 1 组 4 块中的任一块中。

现在说明组相联方式下地址变换。参见组相联方式下的地址变换示意如图 7-35 所示。若在此之前 CPU 曾访问过 28B5000H～28B5FFFH 这块主存块中的某一地址，且需要将该块调入 cache 中。则 cache 系统分析该主存块的区号为 516H；区内的组号为 1；组内的块号为 01。这就确定这一主存块只能装入 cache 第 1 组的 4 块中。

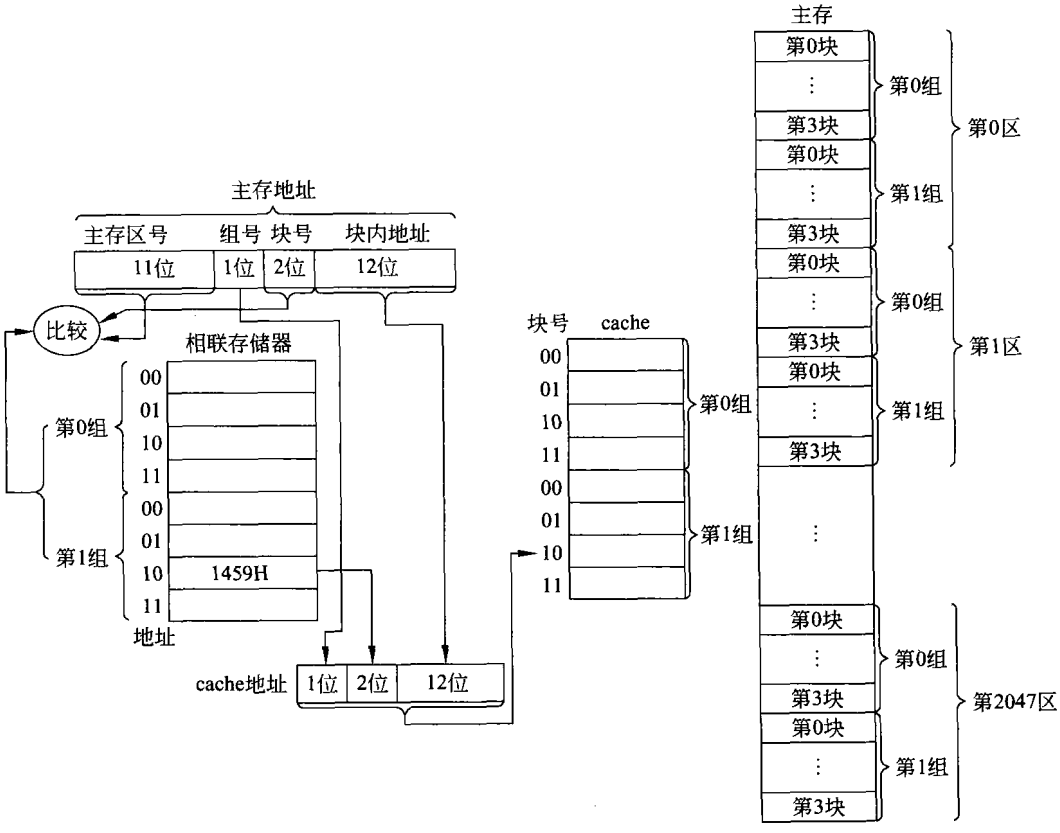


图 7-35　组相联地址变换

在 cache 第 1 组中进行搜索，若发现 10 块空着还未被使用或根据下面的替换算法应将 10 块替换出去。则将主存的这一块装入 cache 的 10 块中。与此同时，还必须在相联存储器相应的第 1 组 4 个单元中寻找地址与 cache 的 10 块对应的 10 单元存放该段的区号 516H 和组内的块号二进制 01，将块号放低位，相联存储单元中应保存的数据为 1459H。此时组号是不需要保存的，因为它是固定的。

若某一时刻 CPU 送出主存地址为 28B57A4H，通过图 7-34 分析可以发现该块一定在第 1 组中，而且由主存区号和组内块号构成的检索字为 1459H。这时，相联存储器只在第 1 组内进行检索比较，发现匹配字所在的地址为 10。此时便可如图 7-35 所描述的那样构成 cache 的地址：地址的最高位就是主存给出的组号(本例中为 1)，接下来是经相联比较得到的地址(本例中为 10)，最后接上主存地址的最低 12 位(块内地址)。这样就构成了 15 位的 cache 地址 67A4H。

从图 7-35 的地址变换中可以看到，在此情况下，相联存储器的容量为 8×13b。每个存储单元除存放主存区号外，还需要存放组内块号。而在相联检索比较时，只需比较主存地址中给出的那一组相联存储单元就可以了。

7.3.2 替换算法

当要访问的主存块未在 cache 中，即未命中时，就需要将主存的块调入 cache，替换 cache 中的块。主要的替换算法有如下几种。

1. 随机替换算法(RAND)

随机替换算法是用随机函数发生器产生需替换的块号，将其替换。这种算法没有考虑信息的历史及使用情况，故其命中率比较低。现已不再使用。

2. 先进先出算法(FIFO)

先进先出算法是将最先装入 cache 的那一块替换出去。这种算法只考虑信息的历史情况而没有考虑其使用情况，也许最先装入的那一块正在频繁使用。因此，该算法也有一定局限性，命中率也不是很高。

3. 近期最少使用算法(LRU)

近期最少使用算法就是将近期最少使用的块替换出去。这种算法需要对每块设置一个计数器，某块每命中一次，就将其计数器清 0 而其他块的计数器加 1，记录 cache 中各块的使用情况。当需要替换时，便将计数值最大的块替换出去。前面提到 cache 的工作是建立在程序执行及数据访问的局部性原理。因此，该算法较前两种算法的命中率要高一些。

4. 最不经常使用算法(LFU)

最不经常使用算法是将一段时间里被访问次数最少的块替换出去。这种算法也需要对每块设置一个计数器，且开始调入时计数为 0。每被访问一次，被访问块的计数器加 1。当需要替换时，便将计数值最小的块替换出去。同时将所有各块的计数器清 0。这种方法将计数周期限定在两次替换的时间间隔内，不能完全反映近期的访问情况。

5. 最优替换算法(OTP)

要实现最优替换算法程序需执行两次。执行第一遍时,记录各块地址的使用情况。根据第一遍的记录就能找出需要替换出去的该是哪块。有了先前的替换信息,在第二次执行时一定能将命中率达到最高。前面的算法是依据过去的信息决定未来,而这种算法是以未来的信息决定未来,必然最佳。

显然,这种方法是不实用的。但它可以用做衡量其他算法的标准。

下面通过一个简单的例子说明上述替换算法。假定程序在主存为 5 块,cache 为 3 块。CPU 执行程序的顺序为:P2、P3、P2、P1、P5、P2、P4、P5、P3、P2、P5 和 P2。

画出 FIFO 算法、LRU 算法和 OTP 三种算法的命中情况如图 7-36 所示。

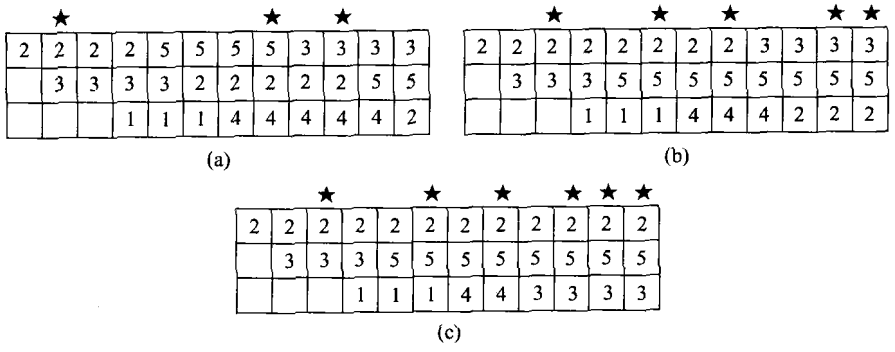


图 7-36 三种替换算法的比较

如图 7-36(a)所示的是利用 FIFO 算法的执行结果。图中用★所表示的是命中的情况,在利用 FIFO 算法的情况下,命中率为 3/12=25%。

如图 7-36(b)所示的是利用 LRU 算法的执行结果。同样,图中用★所表示的是命中的情况,在利用 LRU 算法的情况下,命中率为 5/12=42%。

最优替换算法 OTP 的替换情况如图 7-36(c)所示,在此情况下,命中率为 6/12=50%。

上面的例子仅仅说明 OTP 算法的命中率要高一些。当然,人们期望命中率尽可能的高。

7.3.3 主存——cache 内容的一致性问题

从前面的讲述中可以看到,cache 的内容是主存部分内容的复制,其内容应当与主存的内容保持一致。很显然,只是从 cache 中读出指令或数据是不会改变 cache 中的内容的。而 CPU 对 cache 内容的写就会改变其内容,为了保证主存与 cache 内容的一致性,主存的内容也必须更新。由于程序 cache 是只读的,不存在一致性问题。该问题只发生在数据 cache 中。在那里,如何更新主存的内容,可采取下面所描述的写策略。

1. 写回法

写回法是当 CPU 写 cache 命中时,只将数据写入 cache 而不立即写入主存。只有当被

CPU 写入修改的块被替换出去时才写回到主存中。

如果 CPU 写 cache 未命中，则写修改是将相应主存块调入 cache 之后，在 cache 中进行。对主存的修改仍留待该块替换出去时进行。

实现这种方法需要对 cache 的每一块都设置一位修改标志，以便在该块被替换出去时，根据此标志决定是将该块写回主存还是直接丢弃不要。

这种方法的写修改是在 cache 中进行，很可能会发生多次，而对主存的修改只有最后替换时的这一次。这比后面要说明的全写法速度要快得多。但在替换前时间里存在不一致的问题。

2. 全写法

全写法是当 CPU 写 cache 命中时，在将数据写入修改 cache 的同时写入修改主存，较好地保证了主存与 cache 内容的一致性。

当 CPU 写 cache 未命中时，就直接对主存进行写入修改。而后，可以将修改过的主存块调入 cache，也可以不将修改过的主存块调入 cache。

全写法同时写入修改了 cache 和主存，一致性好，而且也不需要设置修改标志。但是，每次修改都要对速度慢的主存操作，必然降低了 CPU 的运行速度。

7.3.4 cache 性能分析

1. 加速比

前面已经提到，在计算机中设置 cache 的主要目的就在于降低存储系统的价格、提高存储系统的速度。尤其是速度，它是计算机的永恒的主题。因此，人们最关心的问题是加速比。

假设 cache 的访问周期为 T_C，主存的访问周期为 T_M，cache 的命中率为 H，则 cache 系统的平均访问周期 T 可如下表示：

$$T = H \cdot T_C + (1 - H) \cdot (T_C + T_X) \tag{7-1}$$

在式(7-1)中，T_X 为块的调入时间或替换开销，若 cache 以一个字或一个字节(对主存的一次访问可以读写的字长)为一块，则 $T_X = T_M$。若以 n 个字或 n 个字节为一块，则 $T_X = nT_M$。

当命中率很高时，cache 系统的平均访问周期就接近于 cache 的访问周期 T_C。例如，若某计算机 cache 以 1 个字为一块，从 0 地址开始顺序读 10 个字，重复 100 次，则命中率为 99%，假定 T_M 为 100ns，cache 的访问周期 T_C 为 10ns。则可得到 cache 系统的平均访问周期 T 为 11ns。若该 cache 以 10 个字为一块，同样从 0 地址开始顺序读 10 个字，重复 100 次，命中率为 99.9%，T_M 为 100ns，cache 的访问周期 T_C 为 10ns。则可得到 cache 系统的平均访问周期 T 仍为 11ns。

可见，块越大命中率越高，替换开销也越大。但同样的操作平均访问周期 T 没有改变。

cache 系统的加速比 S_P 定义为：

$$S_P = T_M / T \tag{7-2}$$

根据加速比的定义，上面所提到的计算机 cache 系统的加速比 $S_P=100/11=9.9$。

可以看到，在块大小决定后，命中率越高，加速比越大。最大也只能接近 T_M/T_C。就如上面的例子，只有命中率为 100%时才能达到加速比的极限 10，实际上是不可能做到的，只能是希望加速比尽可能大一些。

2. 成本

在本小节开始时曾说明，cache 的出现是为了提高 CPU 访问存储器的速度，同时又可以降低成本。

假设计算机中的主存与 cache 的容量分别为 S1 和 S2。显然，S1≫S2。同时，若主存与 cache 的单位价格分别为 C1 和 C2，而且 C1 一定是低的。

则存储器的平均价格 C 由下式决定：

$$C=(C1S1+C2S2)/(S1+S2) \tag{7-3}$$

尽管 cache 的价格比主存高，但是当其容量很小时，存储器的平均价格 C 接近于主存的价格。

从以上两个侧面可以看到，由于设置了 cache 使 CPU 的访存速度接近 cache 的速度，而使存储器的成本接近于主存的价格。

3. 命中率与 cache 容量的关系

cache 的命中率随着它的容量的增加而增加，其关系曲线如图 7-37 所示。

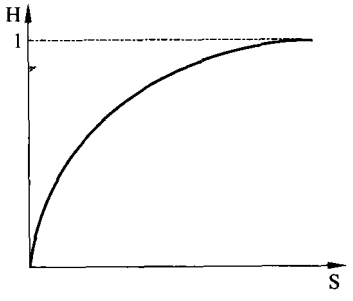


图 7-37　命中率 H 与容量 S 关系

由图 7-37 可以看到，在 cache 容量 S 比较小的时候，随着容量 S 的增加，命中率 H 的增加非常快。随着 cache 容量的增加，命中率增加变得比较缓慢。显然，当 cache 的容量能装下 CPU 要执行的所有程序和数据时，命中率就达到 100%。注意到，当 cache 的容量达到一定量时，再增加容量对命中率的贡献就很有限了。

文献资料中认为，在一般情况下，图 7-37 所示的关系曲线可用下式近似表示：

$$H=1-S^{-0.5} \tag{7-4}$$

式(7-4)中，容量 S 以字节为单位。

4. 两级 cache

为了克服 CPU 和主存间的性能差距，使两者在性能上更好的匹配，cache 应运而生。随着技术的发展，目前均在原有的 cache 和主存之间增加另一级 cache，构成两级 cache 系统。

通常，将第一级 cache 做得比较小，几 KB、几十 KB 到一两百 KB，而且速度很高；而将第二级 cache 做得比较大，一般都在几百 KB 到几 MB，其速度也比较高，甚至与第一级一样。这样一来，当第一级 cache 未命中时，再到第二级 cache 中去搜索。由于第二级 cache 容量很大，其命中率会很高。同时，在速度上也完全可以与 CPU 相匹配。

两级 cache 的总未命中率(或称总失效率)是由两级 cache 的各级失效率来决定：

$$总失效率 = (失效率)_{第一级} \cdot (失效率)_{第二级} \tag{7-5}$$

例如，在 10 000 次访存中，第一级 cache 失效 400 次，第二级 cache 失效 4 次。则可以计算出$(失效率)_{第一级}$为 4%而$(失效率)_{第二级}$为 1%，利用式(7-5)得到使用两级 cache 后总的失效率降为 0.04%。

目前，均将第一级 cache 集成在 CPU 内部(称 L1)，而将第二级 cache 放在 CPU 外部外接(称 L2)。除两级 cache 外，还可采用三级 cache，此处不再说明。

7.4 虚拟存储器

在计算机存储系统中，虚拟存储器是非常重要的。可以说没有虚拟存储器就不可能有性能好的计算机。目前，虚拟存储器存在于所有 PC、服务器等计算机中。

7.4.1 虚拟存储器的概念

前面，通过在存储器系统中增加一个容量小、速度高的 cache，解决了 CPU 与主存储器的速度匹配问题。随着技术的发展，与存储系统有关的另一个问题凸现出来：要求计算机具有海量存储器，以便支持多用户、多任务程序的开发。

目前，不可能将计算机的主存做到海量，而且即使做到了，其价格也是使用者无法承受的。那么，在一定的主存容量下，如何做到让程序员几乎可以无限制地(比主存的实际容量大成千上万倍)使用存储器。解决的办法就是采用虚拟存储器。

虚拟存储器是由价格比较贵、容量不太大、速度相对比较高的主存储器和价格很低、容量非常大、速度慢的外部(辅助)存储器，在操作系统及辅助硬件的管理下，构成了像一个单一的、可直接访问的超大容量的主存储器。

可见，虚拟存储器为程序员提供了比计算机实际主存地址空间大得多的地址空间。它是利用了大容量外存(如硬磁盘)构成的。它解决的是主存容量与价格的矛盾，使速度接近主存速度而容量和价格又接近外存。虚拟存储器的结构形式如图 7-37 所示，虚存是在硬、软件的管理支持下，由主存和辅存共同实现的。

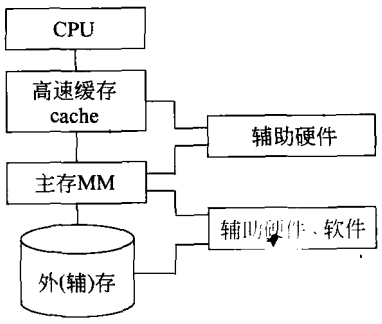


图 7-38 虚拟存储器的构成

可依不同管理方式构成虚拟存储器，下面逐一说明。

7.4.2 虚拟存储器的管理

1. 页式虚拟存储器

(1) 地址变换

在页式虚拟存储系统中，要将虚拟地址空间分成固定大小的页，称为虚页或逻辑页。同时，主存也分成同样大小的页，称为实页或物理页。页的大小为 2 的幂字节或字，一般为几

KB(或 KW)。这样一来,虚拟地址和主存物理地址都包含有两个字段。

对于虚拟地址来说,高字段为虚页号,低字段为页内地址。

对于主存地址来说,高字段为实页号,低字段为页内地址。

在页式虚拟存储器中,由虚拟地址到主存物理地址的变换是通过页表来实现的。在页表中每一条记录都包含有虚页号所对应的实页号。在地址变换中,只要根据虚页号找到对应的实页号,将此实页号与虚拟地址中的低字段所给出的页内地址拼接在一起,就获得了主存的物理地址。页式虚拟存储器地址变换如图 7-39 所示。

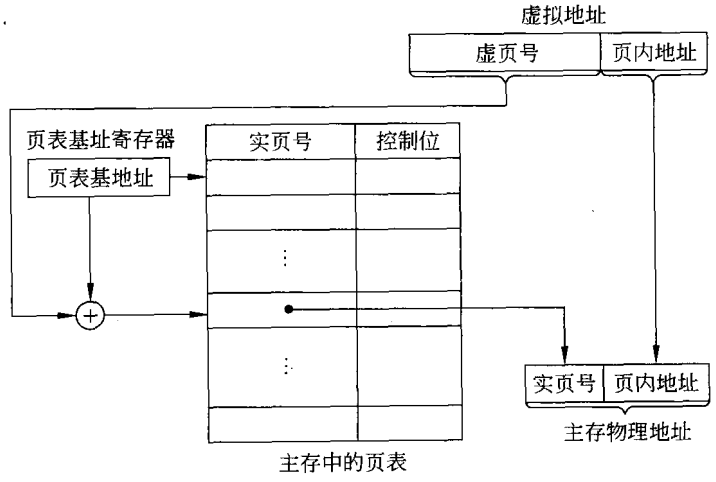


图 7-39 页式虚拟存储器的地址变换

由图 7-39 可以看到,通常页表设置在主存中,表的起始地址可通过页表基址寄存器来设定。当程序给出虚拟地址后,CPU 首先以虚页号为偏移地址查表,从而获得相应的实页号。将实页号与页内地址连接到一起,便是主存的物理地址。

页表中,每一行记录里必须有几个控制位,包括装入位(用以表示该页是否已装入主存)、修改位(用以表示该页是否已被修改)、替换控制位(用以表示该页的使用情况)等。当查表未发现所要访问的页时(未命中),则需按某种算法将要访问的页由外存装入主存。

综上所述,页表的容量一般是比较大的。例如,虚存空间若为 4GB,而页的大小为 4KB,则页表应包含 1M 字,且每字应为一个记录(实页号和控制位)。

(2) 快表与慢表

根据上面描述,由于页表设置在主存中,在页式虚存工作时,首先是要访问主存中的页表进行地址变换获得主存地址。然后,再利用主存地址访问主存获得数据。显然,即使主存命中,也需要两次访问主存才能获得主存信息。前面已提到主存速度比较慢,这必然降低了 CPU 的速度。若是被访问的页不在主存中,那速度就更慢。

为了尽可能提高速度,可借鉴前面 cache 的思路,将页表中最活跃的部分放在 cache 存储器中,构成快表。对快表的查找及管理也全用硬件来实现,以达到快的目的。

快表一般很小,它仅是主存中页表(相对快表可称其为慢表)的一小部分。只有在快表中找不到目标内容时,才去访问慢表,从而提了高速度。利用快表工作的思路如图 7-40 所示。

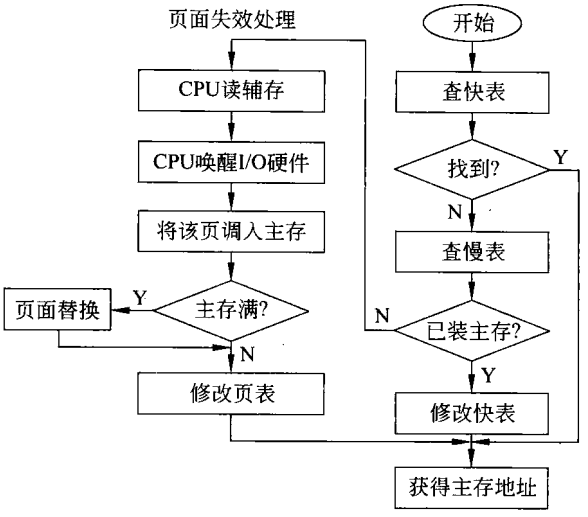


图 7-40 页式虚拟存储器快表工作思路

页式虚拟存储器的主要优点是：①主存的利用率高。由于页的容量很小，每一用户程序只有不到一页的浪费，故形成的主存碎片很小。②地址表的内容比较简单。③地址映像与地址变换速度比较快。

这种方式的缺点：①程序的模块性差。因为页的大小是固定的，无法与程序模块保持一致。②页表很长。如前面的例子可以看到页表长达 1M 字。

2. 段式虚拟存储器

另一种经常采用的虚拟存储器是段式虚拟存储器。采用此种方式是将完成某种独立功能的程序模块定义为一段，例如主程序、子程序、数据块、表格等均可定义成一段。程序员在编程时，每一段都可从虚拟地址的 0 地址开始，并且每一段的长度是不一样的。

在进行段式虚拟存储器管理中，应将每一段程序规定一个段号(也可以定义某一段名)，最方便的办法是各段段号顺序排列，在表中可省略此项。确定每段的长度。同时，在主存中建立一个段表。

段表中每一行由段号、段起始地址、装入位、段长、属性等内容构成。每一行对应一个段。段号是每段唯一的；段起始地址是该段装入内存的起始地址；装入位状态用来表示此段是否已装入主存；段长度表明该段在主存中所占的地址空间；属性用来表征该段的其他属性特征，如可读写、只读、只写等。例如某 8 号段，装入内存的起始地址为 4000000H，该段长度为 64KB，装入位为 1。则意味着该段程序已装在了主存地址 4000000H 到 400FFFFH 的地址范围内。

段式虚拟存储器管理中地址映射的方法是将程序段装入主存的任何可以放的位置上，顺序是随意的。当主存被占满，所剩空间不足以装入需装入的段时，就需依某种替换算法进行替换。

段式虚拟存储器地址的变换如图 7-41 所示。

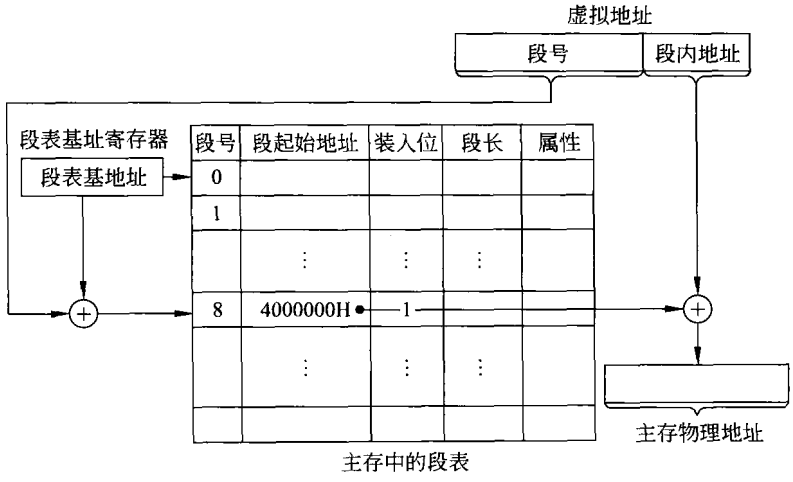


图 7-41 段式虚拟存储器的地址变换

在地址变换中，将虚拟地址所确定的段号为偏移地址查段表，发现该段已装入主存，则直接将起始地址与虚拟地址中的段内地址相加便可得到主存的物理地址。

段式虚拟存储器的优点如下。

(1) 很适合模块化程序设计。将每一功能独立的程序模块定义为一段，既便于程序的开发，又便于虚拟存储器的管理。

(2) 便于程序和数据的共享。此方式能将整段独立的程序和数据装入主存，起始地址、段长度均已知，很容易实现共享。

(3) 便于信息保护。对装入段的属性进行设置，很容易实现对段的保护。

段式虚拟存储器的缺点主要表现在，由于各段的长度是不一样的，在装入主存中分配地址空间比较麻烦，段与段之间可能会产生比较大的空隙(段间的主存碎片)，降低了主存储器的利用率。

3. 段页式虚拟存储器

上面提到的两种虚拟存储器，各有优缺点。为将两者的优点结合到一起，人们提出段页式虚拟存储器。

段页式虚拟存储器是将程序首先分段，段的概念同前。然后，再将每段分成大小相同的若干页。对段来说要用段表来管理所有各段，段表中存放对应该段的页表基地址等有关信息，如图 7-42 所示。每一段有自己的页表，其中存放对应本段每一页的实页号。显然，在装入某一段的各页时，这些页在实存中并不一定连续。

如图 7-42 所示，在进行地址变换时，首先根据虚拟地址决定所在的段号，查段表可查出要访问的本段页表的起始地址。由此地址及虚拟地址中的页号再查本段的页表，从中查出该页的实页号。最后，将实页号与虚存给出的页内地址拼接在一起就获得到主存的物理地址。

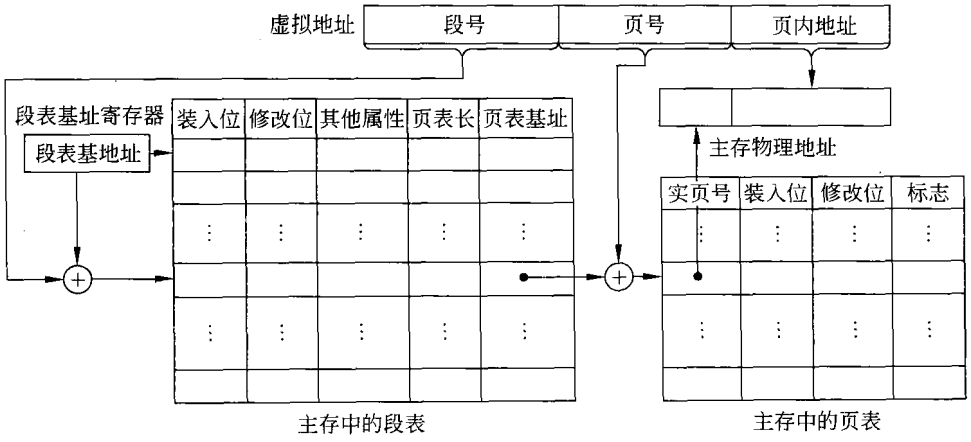


图 7-42 段页式虚拟存储器的地址变换

7.4.3 几点说明

在上面介绍虚拟存储器的三种方式中，为简单起见描述它们的基本原理和概念，尚有几个问题需说明。

1. 多用户工作

前面有关虚拟存储器都是以单用户(仅一个程序员)的情况，即一个程序员在虚拟存储空间里编写许多程序模块。以上的三种方式均可支持单用户的工作。

当计算机系统工作在多用户的情况下，多个程序员各自编写自己的程序，则要求虚拟存储器也能支持多用户的工作。由于虚存空间是海量的，解决多用户的问题完全可以由虚拟存储器管理(操作系统)来指定用户的不同使用空间，即对用户编号。而各用户在编程时可自由使用整个虚拟存储空间，而在虚存管理过程中会为不同的用户编号。这时的虚拟地址较前多出一个字段，如图 7-43 所示的就是段页式虚存地址的管理格式。同样，对段式虚存只是在图 7-43 中无页号字段，而对页式虚存则无段号字段。

虚拟地址

用户号	段号	页号	页内地址

图 7-43 多用户段页式虚拟地址格式

根据图 7-43，在进行虚实地址变换时，首先是以用户号查用户表，从中查到用户段表的基地址，后面的工作如前所述。在段式或页式虚拟存储器中，通过查用户表找到用户段表或页表的基地址，得到基地址后的过程也如前所述。

可见，在多用户的情况下，比单用户要多一步查用户表的过程。

2. 调度方式

在三种虚存管理方式之下，由于主存容量有限，这就存在将页或段装入主存的问题。

段式虚拟存储器是相联地址映射方式，在主存中寻找足够空间将要访问的段装入。若无空位，则需按替换算法将某段替换出去。

在页式或段页式虚拟存储器中，可采用两种方法。

(1) 分页式。即将程序的所有页一次装入主存。在后面程序执行中主存命中率为100%。

(2) 请求页式。这种方法每次只装入一页，用到哪页装入哪页。

显然，两种方法各有优缺点。

3. 替换算法

在虚拟存储器的管理中，同样存在段或页未能在主存命中，需要将新的段或页从外存中装入主存，将主存中的段或页替换出去。

在虚拟存储器的管理中可采用的替换方法与 cache 中的一样，这里不再说明。

习　题

1. 试以单元电路说明，为什么 DRAM 的功耗比 SRAM 小？

2. 某以 8088 为 CPU 的微型计算机内存 RAM 区为 00000H～3FFFFH，若采用 6264，62 256，2164 或 21 256 各需要多少片芯片？

3. 试利用 6264 芯片，在 PC/XT 系统总线上实现 00000H～03FFFH 的内存区域，试画出连接电路图。若在 8086 系统总线上实现上述内存，试画出连接电路图。

4. 叙述 EPROM 的编程过程，说明 EEPROM 的编程过程。

5. 内存按字节编址，地址从 40000H～BBFFFH 共有多少字节(KB)？

6. 试判断 PC/XT 系统中存储系统译码器 74LS138 的输出 $\overline{Y}_0$、$\overline{Y}_4$、$\overline{Y}_6$ 和 $\overline{Y}_7$ 所决定的内存地址范围，如图 7-44 所示。

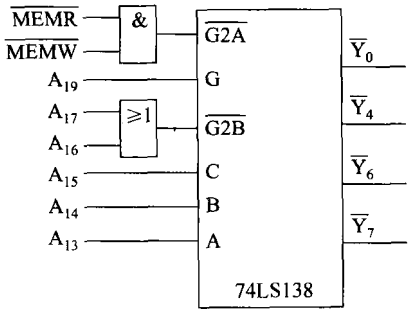


图 7-44　习题 6 附图

7. 将 4 片 6264 连接到 8086 系统总线上，要求其内存地址范围为 70000H～77FFFH，画出连接图。

8. EEPROM 98C64A 芯片的各引脚的功能是什么？如果要将一片 98C64A 与 8088 微处理器相连接，并能随时改写 98C64A 中各单元的内容，试画出连接电路图(地址空间为 40000H～41FFFH)。

9. 与 EPROM 相比，EEPROM 有什么不同？

10. 在 80486 系统中，某 SRAM 芯片容量为 256K × 8b，试用这样的芯片构成 40000000H 到 400FFFFFH 的内存，画出电路连接图。

11. 就书中提到的动态存储器 2164，说明动态存储器的读、写过程。

12. 标准的动态存储器 DRAM 与同步动态存储器 SDRAM 的主要不同表现在哪些方面？

13. DDR SDRAM 的 DDR 是什么意思？与一般的 SDRAM 相比有些什么不同？

14. 半导体动态读写存储器 DRAM 在使用时需要定时刷新，其主要原因是_____。

15. 掉电后，下面说法中正确的是_____。

A. RAM 的数据不会丢失

B. ROM 的数据不会丢失

C. EPROM 的数据会丢失

D. DRAM 的数据不会丢失

16. 某 CPU 地址总线为 A_{19}～A_0，数据总线为 D_7～D_0，内存读信号为$\overline{MEMR}$，内存写信号为$\overline{MEMW}$，某 SRAM 连接电路图如图 7-45 所示。

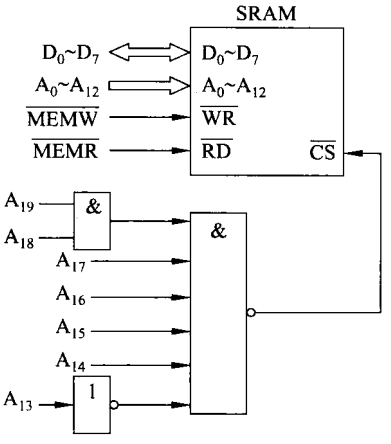


图 7-45 习题 16 附图

(1) 分析该图，请指出该 SRAM 芯片占用的内存地址范围。

(2) 采用该 SRAM 芯片实现 30000H～33FFFH 的内存区域，试画出连接图。

17. 存储器的存取时间 t_A 和存储周期 t_m 的关系是_____。

A. $t_A > t_m$　　B. $t_A \geqslant t_m$　　C. $t_A = t_m$　　D. $t_A < t_m$

18. 主存与 CPU 之间增加高速缓存 cache 的目的是_____。

A. 解决主存与 CPU 之间的速度匹配问题

B. 扩大主存的存储容量

C. 扩大外存储器的寻址空间

D. 提高外部存储器的速度

19. 在计算机中，采用虚拟存储器的目的是_____。

A. 提高内部存储器的速度

B. 扩大内部存储器的寻址空间

C. 扩大外部存储器的寻址空间

D. 提高外部存储器的速度

20. 在计算机中，CPU 对其访问速度最快的是_____。

A. 主存　　B. cache

C. CPU 中的通用寄存器　　D. 硬磁盘

21. 若 cache 以字为块，其存取时间为 10ns，主存的存取时间为 100ns，系统的平均存取时间为 17.5ns，则 cache 的命中率为_____。

A. 96%　　B. 95%　　C. 94%　　D. 92%

22. 高速缓存 cache 与主存间采用组相联地址映射方式(即组间直接，组内全相联)，高速缓存分为两组，每组包含 4 块，每块为 1KB。主存容量为 1MB。

(1) 说明主存区号、区内组号、组内块号、块内地址号各用多少位表示？

(2) 已经地址变换表如图 7-46 所示，若主存地址为 ABCDEH，试确定变换后的高速缓存的地址。

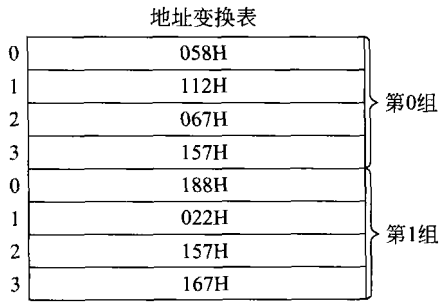


图 7-46　习题 22 附图

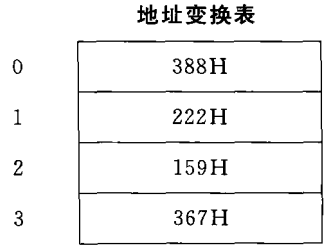


图 7-47　习题 23 附图

23. 高速缓存 cache 与主存间采用全相联地址映射方式，高速缓存的容量为 4KB，分为 4 块每块 1KB，主存容量为 1MB。

(1) 地址变换表为_____个存储单元，每个存储单元包括_____位。

(2) 若主存读写时间为 300ns，高速缓存的读写时间为 30ns，而平均读写时间为 32.7ns，则该高速缓存的命中率为_____%。

(3) 若地址变换表如图 7-47 所示，试根据主存地址确定变换后的高速缓存的地址。

- 主存地址为 88888H 时，高速缓存地址为_____ H。
- 主存地址为 56789H 时，高速缓存地址为_____ H。

(4) 简要说明采用全相联映射的优缺点。(主要从 cache 的利用率、命中率、地址变换硬件的繁简等方面进行说明)

24. 某计算机主存按字节编址，主存与高速缓存 cache 地址变换采用组相联映射方式(即组内全相联，组间直接映射)。高速缓存分为 2 组，每组包含 4 块，块的大小为 512B(字节)，主存容量为 1MB。构成高速缓存的地址变换表相联存储器容量为__①__b。每次参与比较的存储单元为__②__个。

①A. 4×10b　　B. 8×10b　　C. 4×11b　　D. 8×11b

②A. 1　　B. 2　　C. 4　　D. 8

25. 使 cache 命中率最高的替换算法是_____。

A. 先进先出算法 FIFO

B. 随机算法 RAND

C. 优化替换算法 OPT

D. 替换最近最少使用算法 LRU

26. 内存的段式管理有许多优点，下面的描述，_____不是段式管理的优点。

A. 支持程序的模块化设计和并行编程的要求

B. 各段程序的修改互不影响

C. 地址变换速度快、内存碎片(零头)小

D. 便于多道程序共享主存的某些段

27. 若 cache 容量为 100 字，并以 50 字分块，起始为空，CPU 从主存单元 0,1,2,…,99，依照一次读出一个字，顺序读出 100 个字，并重复读 10 次，求命中率为多少？若主存的速度比 cache 慢 5 倍，则在此情况下比无 cache 速度提高多少倍？

28. 主存容量为4MB，虚存空间为1GB，实地址与虚地址各为多少位？若页面大小为4KB，则主存页表应有多少个表目？

29. 段式管理虚拟存储器段表如表7-3所示。

表7-3 虚拟存储器段表

段号	段首地址	装入位
0	50000H	1
1	30000H	0
2	15000H	1
3	44000H	1

若访问段3的1059H单元和段2的1678H单元，它们对应的实地址各是多少？

30. 某页式管理虚拟存储器共有256K页，每页为4KB。主存容量为1MB。试问实存分多少页？主存页表有多大？描述虚实地址的变换过程并举例说明。

31. 若某程序运行要求页面的顺序为P3、P4、P2、P6、P4、P3、P7、P4、P3、P6、P3、P4、P8、P4、P6，主存容量为3页，程序运行前主存为空。求FIFO算法和LRU算法的命中率。若主存增至4页，再求各自的命中率。